

POLITECNICO
DI TORINO

this:

POLITECNICO DI TORINO

Master Degree course in Software Engineering

Master Degree Thesis

Integration of Intelligent agents in Game Development: Prompt Engineering Methodologies for Automatic code generation

Supervisors
Prof. Riccardo Coppola

Candidate
Francesco Re

Academic Year 2025-2026

Acknowledgements

Before beginning this paper, I would like to dedicate a few lines to those who have supported and accompanied me throughout my personal and professional growth.

I would like to express my sincere gratitude to my Supervisor, Riccardo Coppola, for his practical support, expertise and guidance.

My heartfelt thanks go tho my family, especially my parents and my sister, for their teachings and for always encouraging me to pursue my future goals.

Finally, I want to thank my friends for sharing the joys, successes and challenges of both daily life and my university journey.

Abstract

The integration of Artificial Intelligence (AI) within software engineering has significantly expanded following the advent of Large Language Models (LLMs) and natural language processing. These technologies have been implemented across a vast array of techniques to optimize developer workflows, a trend that is equally prominent in game development.

While current research predominantly focuses on generating complete, end-to-end experiences via prompts and agentic architectures, this study explores the use of LLMs as tools to enhance a developer's architectural design, effectively translating creative intent into technical implementation.

Utilizing prompt engineering techniques, this research aims to develop a short game project featuring exploration and a turn-based combat system. The project was built using the Unity engine, with most game object components generated entirely through AI code generation. The methodology relied on two main approaches: detailed file specifications and a multi-level prompting strategy that progressively narrowed the scope toward full class implementation. Once the initial classes were generated, a Human-in-the-Loop (HITL) approach was adopted to integrate additional mechanics and resolve anomalies. To execute this framework, local models were utilized, specifically Qwen3-coder 30B.

The empirical results indicate that across 149 prompts used to create 24 classes, these strategies generated code that rarely exhibited syntax errors, aligned consistently with the initial design, and largely avoided hallucinating divergent functions. Each class required an average of 12 prompts (with a median value of 4). The majority of the prompts (96) were used within the HITL phase to refine results, while the remaining 53 prompts were dedicated to class building and implementing new features.

However, this methodology revealed constraints in specific areas. Hallucinations were not entirely eliminated, averaging 0.88 per class (with a median of 0). Furthermore, longer implementations degraded model efficiency, occasionally leading to corrupted outputs. Finally, the strict constraints of the prompting framework made the final implementation heavily dependent on the initial design, leaving the model with little autonomy to correct foundational flaws in the architecture.

Overall, this study demonstrates that the proposed framework enables the creation of game projects with minimal programming language expertise, shifting the developer's core requirement toward critical thinking and design validation.

Contents

1 Introduction 5
- 1.1 Overview of the content . . . 5
- 1.2 Problem definition and background introduction . . . 5
- 1.3 Software engineering: a taxonomy of methodologies . . . 6
  - 1.3.1 Waterfall methodology . . . 6
  - 1.3.2 Agile methodology . . . 7
- 1.4 AI in software development: an historical and methodological view . . . 9
  - 1.4.1 The Evolution from Symbolic Rules to Connectionist Paradigms . . . 9
  - 1.4.2 The Precursors to Transformer Architectures . . . 10
  - 1.4.3 The Attention Revolution and the Transformer Architecture . . . 12
  - 1.4.4 The Bifurcation of Language Modeling Paradigms . . . 14
  - 1.4.5 Scale, Emergent Abilities, and the Scaling Laws . . . 15
  - 1.4.6 Alignment, Instruct Models, and the Contemporary State . . . 16
  - 1.4.7 Agentic architecture: an enhanced LLM working strategy . . . 17
- 1.5 Prompt engineering: shaping the LLM's work . . . 18
  - 1.5.1 Core Prompting Taxonomies and Methodologies . . . 19
- 1.6 Game development . . . 22
  - 1.6.1 Pre-production phase . . . 22
  - 1.6.2 Production stage . . . 22
  - 1.6.3 Unity engine: the industry SOTA . . . 24
  - 1.6.4 Post-production stage . . . 26
  - 1.6.5 The integration of AI in the process . . . 26
  - 1.6.6 Prompt Engineering in Game Development . . . 27
  - 1.6.7 LLM's technical limitations . . . 29
  - 1.6.8 Ethical doubts and challenges . . . 30

2 System model 31
- 2.1 The scope . . . 31
- 2.2 Related work . . . 31
  - 2.2.1 MarioGPT: GPT-2 fine-tuned as level generator . . . 31
  - 2.2.2 The expansion of the paradigm: GameGPT, an agentic architecture built to make games . . . 32
- 2.3 The limitations of the approach . . . 34
- 2.4 Shift in paradigm: from whole experience architect to specific component constructor 36
- 2.5 Model description . . . 38
  - 2.5.1 Study approach definition: Generative AI as a Design Translator . . . 38
  - 2.5.2 Shifting Focus from Code Writing to System Engineering . . . 38
- 2.6 The main challenges . . . 39
  - 2.6.1 Mitigating Hallucination and Architectural Drift . . . 39
  - 2.6.2 Preserving Architectural Integrity in a Game Engine . . . 40
  - 2.6.3 Integrating the framework into a normal work context . . . 40

3 My main contribution 41
3.1 The MonoBehaviour Class: Component Composition and the Native-Managed Boundary . . . 41
3.2 The ScriptableObject Class: Serialization, Memory Optimization, and Flyweight Data Design . . . 43
3.3 Structural Synthesis: The Separation of Logic and Assets . . . 44
3.4 Local Inference Infrastructure and Contextual Workspace Integration . . . 46
3.4.1 Ollama: the base of the self-hosted strategy . . . 46
3.4.2 From Tongyi Qianwen to Qwen3-Coder: An Open, Code-Generation Focused Model . . . 49
3.4.3 Interfacing with the model: from a chat-based to an advanced IDE-integrated experience . . . 50

4 Experimental/numerical evaluation 55
4.1 Methodology . . . 55
4.1.1 Environment and actors definition . . . 55
4.1.2 Multi-layer prompt generation phase . . . 57
4.1.3 Logic validation and correction . . . 59
4.1.4 Case study specific setting . . . 59
4.2 Numerical results . . . 62
4.2.1 Evaluation of the empirical results . . . 63
4.2.2 Empirical results . . . 64
4.2.3 Main criticalities . . . 66

5 Conclusion 69

Bibliography 73

Chapter 1

Introduction

1.1 Overview of the content

This document will have the following structure:

- **Chapter 1** will layout a base for the concept used throughout this study. The main topics will revolve Software engineering, AI's history and use and the concept of Game Developement
- **Chapter 2** will present different studies to tackle the challange of AI-powered game development and will introduce the approach used in the context of this study
- **Chapter 3** will describe the technologies used in this research
- **Chapter 4** will show how the research work has been carried out and what notable result have been observed
- **Chapter 5** will serve as conclusion to evaluate the process done, the possible improvements to the methodology and any possible consideration that can be drawn

1.2 Problem definition and background introduction

Within the domain of software engineering, and specifically throughout the lifecycle of game development, a distinct disconnect often emerges between the conceptual engineering phase and the subsequent technical implementation. While the former involves high-level problem-solving and architectural planning, the latter is frequently characterized by the mechanical translation of these abstract designs into functional code. This 'translation' phase, though essential, often introduces a disproportionate amount of overhead compared to the creative engineering process.

The primary challenges in software creation stem from the rigorous abstraction required to map complex, real-world scenarios into discrete variables and conditional logic. Consequently, the coding process serves as a formalization step, where the engineer's mental model is transcribed into an executable format-a process that is often as rigid as it is time-consuming. The consequential coding process is then an application of the previous thought process to "translate" the results of the engineering into an executable format.

In game development, this implementation bottleneck is further exacerbated by the specific nature of modern game engines. The reliance on highly specific method and specialized APIs-particularly within proprietary frameworks-often leads to significant development delays. Unlike open-source environments, proprietary systems frequently lack the robust community-driven documentation and peer-to-peer feedback loops necessary to streamline troubleshooting. This isolation forces developers to navigate complex, opaque systems, turning what should be a straightforward implementation into a source of technical debt and project stagnation.

The scope of this study will be to evaluate the capabilities of using LLM-based approaches in order to enhance the capabilities of a software engineer in the context of generating the code that implements the engineer's design rather than using the models to solve the implementation process.

To do so this study involves the use of a framework used to create a short demo of a game. This small game will be a turn-based experience with some exploration elements.

But before evaluating the nature of the approach used three main topics need to be explained. The first will be about software engineering-the process of studying and creating software. This section will describe the general flow of the process and two alternative frameworks used in the work context.

Consequently the next topic will be about the use of artificial intelligence (AI) in the context of software engineering, starting from how this technologies birthed and evolved to reach today's heights.

Finally the last topic will be about Game development-the process of developing video games-explaining how the process works and how the LLMs can be integrated in the process.

1.3 Software engineering: a taxonomy of methodologies

When considering the Software Development Life Cycle (SDLC) a first concept must be considered before starting evaluating the project, which is the methodology of project management during the code design and implementation.

This focuses exclusively on how the project should be managed on high level rather then the technologies implied in the project. The way a project is handled influences not only the workflow of the team during the coding phase, but also how the final product is presented and can be commercialized.

1.3.1 Waterfall methodology

One of the first approaches ever used is called the waterfall methodology [4].

This approach focus is centered mostly on the concept of linearity, which practically means starting a phase only when the previous is completed. This approach follows the idea of "measure twice, cut once", which means that most of the project research and estimations are done upfront. The usual type of project managed with this methodology are large scale, with pretty clear requirements-which have little to no need of changes. This methodology is used to attain an high level of confidence of receiving a certain outcome.

The workflow of this methodology is divided in the following phases:

- **Requirements**, the fundamental phase of the process. It stems from the idea that requirements can be collected upfront, so this first step focuses mainly on documentation to evaluate not only the requirements but also the costs, assumptions, risks, dependencies and metrics needed. Once this phase is ended, a full plan is created and the team has the full picture of how the project is going to be managed.

- **Design**, the translation phase of the process. Given the requirements from the previous phase-which also consider aspects like use scenarios, layouts, and data models-it is the part of the process which focuses on design a possible solution to these problems. This process is divided into high-level design-in which the general structure of the project is built so that the information access and functionality are defined-and low-level design-in which an implementation for every component of the high-level design is defined.

- **Implementation**, the programming phase. Probably the shortest phase, in which the code base is written with some occasional testing. If significant changes become necessary, it might be required to rollback to the previous stage.

- **Verification or testing**, the stage that confirms that everything has been done properly. Starting from the documents from the requirements and design phases a test suite is built to ensure that no errors on the requirements have been made and that good user experience is given.

- **Deployment and maintenance**, phase that starts with the delivery of the software. Once deployment is done, it is followed by defect detection and carry out change requests made by users.

This kind of approach has clear advantages in terms of management of the whole workflow and cost estimation. Since all the measurements and design are done beforehand design flaws can be detected early and the team knows what are the steps in order to achieve the completion of the project, so even developers who join the project later on can easily catch up. However these strengths are due to a structure that is incredibly rigid, since it does not allow big changes in design. Furthermore the need of constant documentation can become a bottleneck not only because it is time-consuming, but it also makes difficult to perform changes-making the customer not particularly involved in the project process.

1.3.2 Agile methodology

To address the extreme rigidity an alternative SDLC became vastly used in modern workplaces, the Agile methodology [3].

While Waterfall methodology's focus on a linear and hierarchical workflow, this methodology focuses on an iterative and incremental approach with high adaptability-having the capability of changing the priorities in the shortest amount of time possible in order to accommodate the client's need-having a team of equals.

Unlike waterfall methodology, which encompasses the work all at once subdivided into linear phases, this methodology is formed by iterations called sprints. These iterations are brief in time-typically between two weeks and two months-and allow the production of a certain number of features. The focus of the sprint is not the quantity of code written or design made but the amount of functionality that are "done", which means coded and tested so ready to be deployed.

During the sprint four parties are at play:

- **Product owner (PO):** usually represented by the owner, defines the needs and the priorities of the project.

- **Scrum master:** follows the sprint advancement and solves issues.

- **Team members:** work on the implementation of the project.

- **Stakeholders:** give feedback on the work of the sprints.

Another key difference from the previous methodology is the way in which the process is documented. While previously the documentation phase was a necessary step, now the scope is to avoid spending too much time on it, leaving it to an ending stage mainly for maintenance reasons. Instead of defining each functionality through a detailed documentation in this SDLC the description is made through stories and actors. While actors are external parties that interact with the system, stories are definitions of simple functionality that an actor would want to obtain a specific gain. The usual structure is: "As an **Actor**, I want to **Required functionality**, so that **a benefit is obtained**.".

The story must be INVEST-independent, negotiable, valuable, estimable, small, testable-which means that it must be a feature small enough to be made within a sprint, that can be changed with the PO and can create value to the user.

The core of this methodology is described with the so called "Agile Manifesto", which serves as a guiding plan to follow during its adoption. In it, core values are defined, which are:

1. **Individuals and interactions over processes and tools:** great focus is given to the human factor. While technologies are an important component of the process, over-reliance can diminish the adaptability of the team.

2. **Working software over comprehensive documentation:** in the context of a working environment, the creation of functional code has higher importance than creating a full fledged documentation. This value focuses on reducing the amount of unnecessary paperwork done by the developers.

3. **Customer collaboration over contract negotiation:** given the changing nature of the customer need a constant exchange of feedback throughout the process is an invaluable asset.

4. **Responding to change over following the plan:** in the context of change the capability of adapting welcoming any possible change of plan is the key to correct any possible error or deviation from the original path.

Added to this 12 additional principles are presented to guide during the production process. The focus of these principles is toward simplicity, consistency and quality of cooperation between individuals of the development team, allowing continuous production of software with high flexibility to change.

The advantages of this approach are mostly found at individual level, in fact employees has more freedom to work based on their strengths enabling not only a better adaptability and efficiency but also with a better efficiency. However the benefits are found in an environment with strong collaborative capabilities, since it requires the team to self organize the work. In addition given the way this methodology work the results attained can be difficult to predict, potentially causing the loss of direction during the process. This framework is more compatible with project that may have ambiguous requirements or results, which need more flexibility to handle.

The two presented framework answer two different types of needs and team compositions. While the sequential and linear structure of the waterfall model allows an easier way to organize the team and a more stable project to work on, the Agile methodology allows faster deployment, decreases the documentation overhead and enables an increased flexibility not only to change the design but also to improve the consumer's involvement through feedback. (Tab 1.1)

	Waterfall	Agile
Project workflow	Linear, each phases starts after the end of the previous	Iterative and incremental, every cycle repeats all the steps to add functionalities or to apply changes
Planning	Done extensively in early stages, errors in design may cause enormous damages	Done throughout the whole process mainly at the beginning of the cycle, high capability to adapt to new environment
Team organization	Fixed roles doing different tasks	Each developer is equal and capable to organize their own work as wanted
Delivery	Once at the end of the process, finished product	Multiple releases of software, continuously growing product
Best suited for	Projects with stable and clear requirements	Ambiguous project which require continuous change and updates

Table 1.1: Comparative table between the two methodologies

1.4 AI in software development: an historical and methodological view

While being a technology that become popular to the wide public in the recent year, Machine Learning is a subject that has been studied for several decades. In order to understand the methodologies that are going to be used, knowing how these technologies originated and how each technology is based of is key.

1.4.1 The Evolution from Symbolic Rules to Connectionist Paradigms

Prior to the dominance of statistical and deep learning methodologies, the field of artificial intelligence was governed by the symbolic paradigm-primarily manifested as rule-based and expert systems.

Originated around 1960, rule-based systems are structures that build the behavior of the system around set of rules that represent of the knowledge needed. Instead of using a declarative approach, this system is based on a structure of rules that define what the machine does or can conclude based on a specific situation. To do so the rules are expressed in a simple set of assertion, typically through if-then statements. The model can either be starting from the initial conditions and use the rules iteratively until the conclusion is reached-approach formally named forward chaining systems-or initiating from an hypothesis going backwards to prove it, creating new sub-goals as the analysis proceeds-system named backward chaining systems.

The whole structure of the system is made of the rule set, a set of "facts"-assertions which can be seen as collection of data and conditions used to describe the current situation the system is in-and an interpreter-which acts following the needed rule given the facts. To define the rule structure, systems capable of mimicking the human thought process are required so that abstractions about the functionalities of the system can be formalized [15].

It is this main idea-the concept of reproducing the performance of human experts to try to solve problems-that work as a base for the concept of Expert Systems. Instead of trying to formalize the problem in a state space an expert makes shortcuts by employing "thumb rules"-behaviors, knowledge or skills acquired thorough experience. For this reason rather than procedural approaches the main way of working with this types of system is through modular, rule-based methodology. The point of this approach is to use a superficial way to link a fact with a specific outcome/action, without having specific knowledge of the connection's nature

between the two [41].

While highly deterministic and interpretable, rule-based systems suffered from brittle execution when exposed to the inherent ambiguity and fluid syntax of natural language. They were incapable of generalization, requiring human engineers to manually program every edge case and contextual nuance.

This computational bottleneck eventually led to the "connectionist" shift, transitioning the studies from explicit programming to statistical learning. The shift was brought by the concept from the cognitive science of connectionism-the theory of information processing. This system rely on parallel processing of sub-symbols, using the structure of the brain as reference replicate the behavior in an artificial mechanism. To do so the base building block used is the neuron, which act as receiver and manipulator of signals [28]. Instead of structuring rigid logical rules, researchers began utilizing artificial neural networks (ANNs) capable of learning implicit representations directly from data.

This evolution fundamentally altered the objective function of natural language processing: rather than parsing sentences through static grammatical rules, models began optimizing numerical weights across deep layer architectures to minimize prediction loss, laying the structural and philosophical foundation for modern deep learning.

1.4.2 The Precursors to Transformer Architectures

The evolution of modern Large Language Models is rooted in the broader field of Natural Language Processing (NLP)-which is the study of employing machines to analyze and manipulate human language-and statistical language modeling. These methodologies allowed not only to extract the features from text queries, but also to evolve from previous architectures and do incrementally complex tasks.

Early methodologies relied heavily on Markov chains and N-gram models, which computed the probability of a word by evaluating a window of words preceding. These statistical approaches suffered fundamentally from the curse of dimensionality. They were in fact incapable of understanding long-range contextual dependencies. Additionally another critical aspect was their difficulty in spotting semantic similarities of similar words, which didn't appear together in the training phase.

The introduction of continuous word embeddings, most notably Word2Vec in 2013, defined a critical paradigm shift. This shift is mainly represented by the encoding of each word into a vector space. While initial representations were much simpler implementations, like the one-hot encoded vector approach-which encoded one different word to one specific dimension of the vector-, the real first jump in quality was made employing a "positional" approach-where each vector used as encoding correspond to a specific position in the state space. By mapping words into a continuous vector space $\mathbb{R}^d$, geometrically near vectors represented semantically similar concepts. The use of this concept allowed neural networks to finally process language using distributed representations.

To handle sequential data and capture context, the field transitioned to Recurrent Neural Networks (RNNs).

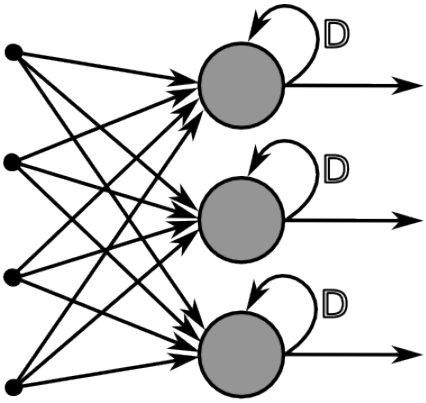


Figure 1.1: RNN's structure [11]

The architecture of the RNNs model is formed of three layers, which are input, recurrent hidden and output layers. The peculiarity of RNNs is the presence of one or more feedback loops-backward links using the output of a layer as input for either a previous layer or the same that produced the results. Using this structure allows evaluation of the results along with the new input, having the system "remember" the context, consequently improving the resulting accuracy to spot patterns.

Additionally to improve the learning process Activation functions are used. Using only linear layers would make multiple hidden layers act as one linear layer, limiting the reach of the network to create connection. To avoid this nonlinear functions are used, allowing to reach nonlinear boundaries. Some of the most used include the "sigmoid" (1.1), the "tanh" (1.2) and the rectified linear unit (ReLU, 1.3).

These function are defined as:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{1.1}$$

$$\tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}} \tag{1.2}$$

$$\mathrm{ReLU}(z) = \max(0, z) \tag{1.3}$$

In order to initialize the model used a first learning step phase, called the "training phase", must be carried out. In this of the process the weights of the network are set, making the machine grasp the relationships within the dataset. To evaluate the results and guide the training phase it is used a loss function-a function that compares the obtained result with the corresponding target. Some common loss function are Euclidean distance and Hamming distance.

Once a loss function is decided, the next choice that must be made is the selected learning method-the way the model improves its internal weight configuration to minimize the loss. One popular and simple approach used is the Gradient Descent (GD). In this methodology the change of weights are done in proportion to the derivative of the error to the respect to that weight, requiring that the non-linear function must be differentiable.

Given that RNN is a structure through time, an extension of GD through time is also needed while training the network. This process is called back-propagation through time and it consists in unfolding time iterations into layers, obtaining an equivalent feedforward network.

This mechanism however has critical issues handling dependencies as the duration increases. In some cases the long-range dependencies tend to "disappear", creating what is called the

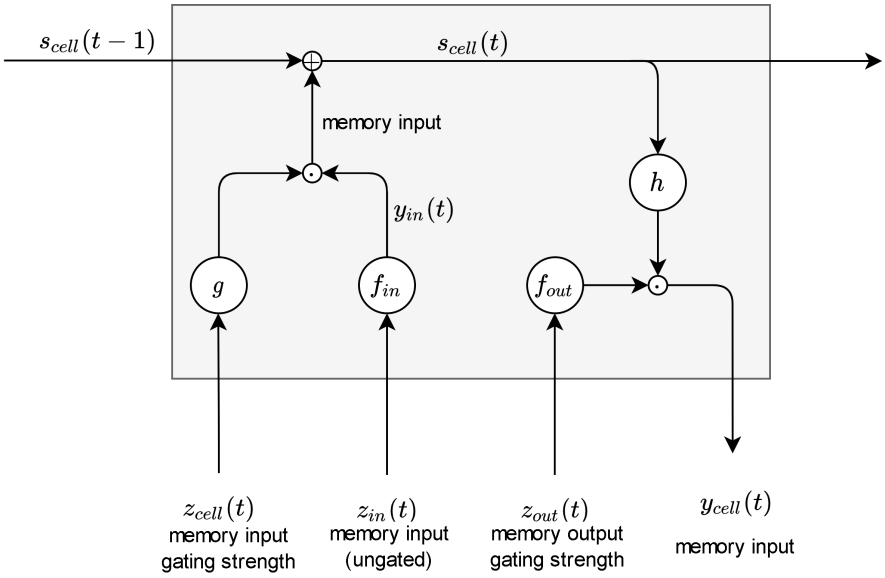


Figure 1.2: LSTM architecture structure [32]

vanishing gradient problem-the exponential decay of the gradient. On the other hand, the back-propagated gradient can blow up, resulting in an unstable learning and creating what has been known as the exploding gradient problem.

In order to mitigate the vanishing gradient problem, Long Short-Term memory (LSTM) architectures introduced specialized gating mechanisms designed to regulate information flow. This approach changes the hidden layers from nonlinear operators to memory units controlled by gates. The gates control the flow of information to hidden neurons and preserve previously extracted features.

Despite their architectural improvements, LSTMs possessed a fundamental engineering limitation: sequential dependency. Because the computation at time step t strictly required the hidden state from time step $t-1$, the training process could not be parallelized across modern GPU architectures. This computational bottleneck strictly limited the size of the networks and the volume of text corpora they could evaluate in a feasible time frame [38].

1.4.3 The Attention Revolution and the Transformer Architecture

The previously mentioned architectural bottleneck was completely dismantled in 2017, with the publication of the seminal paper "Attention Is All You Need" by Vaswani et al. This work introduced the Transformer architecture, which discarded recurrence entirely in favor of the self-attention mechanism-an attention mechanism capable of evaluating the importance of all the tokens simultaneously.

The Transformer architecture operates on an encoder-decoder framework that splits the inference pipeline into comprehension of the input and generation of the output. The process begins with the encoder, which takes the raw input sequence and maps it into a dense, high-dimensional symbolic representation-a matrix of vectors that leverages self-attention mechanisms to capture the deep contextual relationships, syntax, and meaning of the source data. This rich contextual map obtained in this processed is then passed to the decoder, which references the encoder's representation at every step to generate the final output sequence. This happens by ensuring that each newly produced word aligns precisely with the semantic intent of the original input.

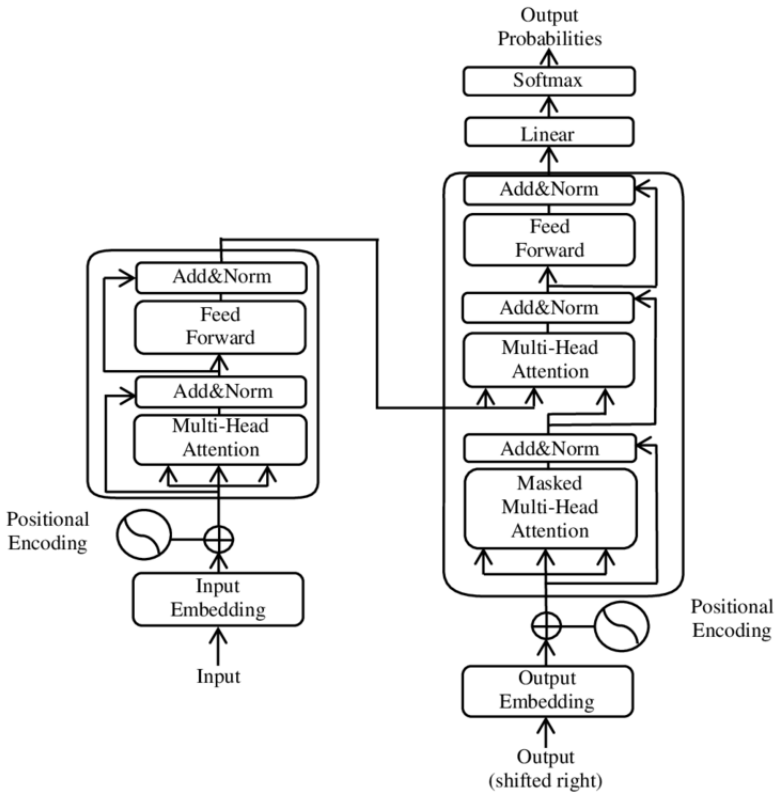


Figure 1.3: Transformer Architecture [21]

The core innovation of the Transformer architecture lies on a specific mathematical operator, called Scaled Dot-Product Attention. By computing dynamically the relevance of every token in comparison to all others it is achieved the complete bypass of the physical distance constraints inherent to traditional recurrent models.

This mechanism operates on three distinct matrices-Queries (Q), Keys (K), and Values (V)-where the output is a weighted sum of the elements in the values set. Each weight is determined by a compatibility function (a dot product) between the query and its corresponding key. To prevent large values of d_k (the dimensionality of the key vectors) from pushing the softmax function into regions with vanishing gradients, the dot products are scaled down by a factor of $\sqrt{d_k}$ before applying the softmax activation, a relationship formalized by the equation:

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{1.4}$$

Rather than relying on just one attention function that averages out complex relationships, the architecture employs Multi-Head Attention, which linearly projects the queries, keys, and values h times using different, learned linear projections. This allows the model to execute the attention mechanism in parallel across h distinct representation subspaces, generating independent d_v-dimensional outputs that are ultimately concatenated and projected once more to produce the final output matrix. By employing multiple heads, the model avoids the information bottleneck of a single attention head-where averaging inhibits the ability to focus on different positions simultaneously. It instead gains the capacity to jointly attend to diverse syntactic and semantic features at different positions across the sequence [44].

By entirely eliminating the sequential, step-by-step dependencies inherent to traditional recurrent neural networks, the Transformer architecture gave the opportunity of parallelization

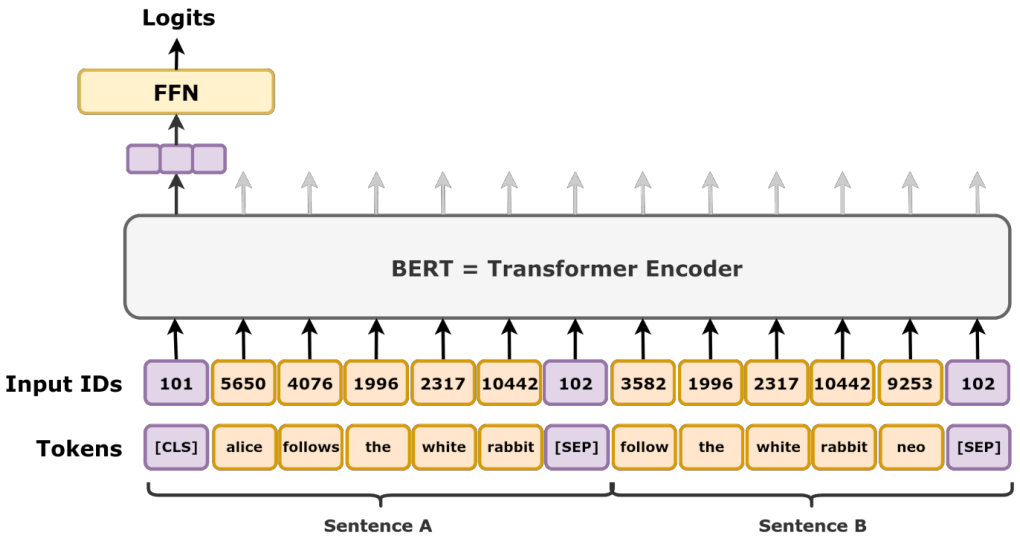


Figure 1.4: BERT Architecture performing NSP task [14]

across massively distributed GPU clusters, allowing entire sequences to be processed simultaneously rather than token by token. This fundamental shift in computational efficiency empowered researchers to increase the scale of both network depth and training dataset sizes by multiple orders of magnitude, effectively eliminating the hardware bottlenecks that had previously constrained deep learning progress.

1.4.4 The Bifurcation of Language Modeling Paradigms

While the original framework introduced in the "Attention Is All You Need" paper utilized a symmetrical encoder-decoder structure tailored specifically for sequence-to-sequence machine translation, the AI community quickly realized the potentialities that could be obtained by isolating the two constituent modules. This realization fractured the evolutionary tree of large language models into two primary branches: encoder-only architectures and decoder-only architectures-which leverage autoregressive token generation to tackle generative and conversational AI applications.

Encoder-only

Also known as autoencoding architectures, this branch of technology specializes in the optimization of bidirectional context extraction for comprehension tasks like classification and information extraction. The epitome of the paradigm in natural language processing (NLP) was established by Google's introduction of Bidirectional Encoder Representations from Transformers (BERT).

Unlike traditional autoregressive models that process text sequentially in just one direction, BERT leverages a deeply bidirectional architecture. This design puts the model in the condition of adjusting each token on its joint left and right context across all layers simultaneously, fundamentally shifting how semantic nuance and contextual dependencies are captured.

Consequently, BERT has demonstrated robust efficacy across a multiple downstream computational tasks, including natural language inference, semantic textual similarity, text segmentation, sequence classification, named entity recognition, and sentiment analysis. The architecture's foundational performance relies on its self-supervised pre-training regime, which is explicitly engineered around two core objectives: Masked Language Modeling (MLM) and Next Sentence Prediction (NSP).

During the MLM tasks a sequence text is presented, in which a random subset of the input tokens-typically 15% of the tokens-are corrupted by replacing them with either a special masking or a random token. The model is then tasked of reconstructing these omitted tokens based solely on the non-masked tokens.

To capture coherence and binary discourse relationships along multiple sentences the NSP (Fig. 1.4 objective tasks the model on the evaluation of sentence pairs (A, B). The network must predict whether sentence B is logically sound as successor to sentence A, labeled as IsNext, or a random sentence sampled from the corpus, labeled as NotNext. Together, these dual pre-training strategies equip BERT with both fine-grained lexical understanding and macroscopic structural awareness of language [5].

While exceptionally powerful for discriminative tasks, encoder-only models are not suited for open-ended, continuous text generation jobs.

Decoder-only

In parallel to the autoencoder approach's progresses other parties pursued the autoregressive approach, the decoder-only paradigm. One of said parties was OpenAI, whose popularity started with the introduction of the Generative Pre-trained Transformer (GPT-1) in 2018.

Autoregressive language models operate under a causal masking constraint, meaning they generate text sequentially by predicting the probability distribution of the next token x_{t+1} based exclusively on the historical context of previous tokens $\{x_1, \ldots, x_t\}$. This architectural restriction ensures that the model cannot look ahead into future token positions, aligning it intrinsically with generative workflows and open-ended text synthesis.

The primary breakthrough of the Generative Pre-trained Transformer (GPT) lineage, however, was due to its validation of a decoupled, two-stage learning paradigm. This framework starts with a massive, unsupervised pre-training phase executed on expansive, general-purpose text corpora. During this stage, the network engages in self-supervised learning directly from raw, unlabeled text. The main objective of this step revolved around the optimization of a standard language modeling to internalize syntactic structures, world knowledge, and long-range dependencies without human annotation. During this phase the model was pre-trained on approximately 4.6 GB of raw textual data extracted mainly from the BookCorpus dataset, a collection of thousands of unpublished novels spanning diverse genres that provided the continuous, narrative-rich text necessary for capturing complex linguistic patterns

The next stage involved a discriminative, supervised fine-tuning (SFT) phase, where the pre-trained weights are minimally modified and adapted to execute specific downstream tasks using localized, labeled datasets. The seminal iteration of this series, GPT-1, was structurally characterized by 117 million parameters [46]. In the context of deep learning, these parameters represent the internal learnable weights and biases-specifically the multi-head attention matrices and feed-forward network weights-that the model dynamically calibrates via gradient descent to map input tokens to target predictions.

1.4.5 Scale, Emergent Abilities, and the Scaling Laws

Between 2019 and 2021, the field entered the era of hyper-scale.

OpenAI's GPT-2 was one of the first following this path, which expanded the scale from the previous model to a 1.5 billion parameters model. This version was born as a "scaled up" version of the predecessor, which was trained with a custom corpus called WebText. To create this 40 GB high quality text training base a scrapping procedure was performed, using the platform Reddit as source. Rather than taking all the possible post, an heuristic was decided. To avoid including worthless data the process was limited to all accounts which had at least 3 karma points-an internal score that measures the quality of a contributor. This metric is based on how a user's response is viewed, indicating if they are evaluated as interesting or entertaining.

After the scraping step a final cleanup operation was performed to guarantee high value of the resulting dataset [33].

The increase in dimensions demonstrated that scaling the model size and data volume allowed the network to perform zero-shot task-where no example of input-output couples is present-transfers without any structural modifications or fine-tuning [18].

This trajectory culminated in the release of GPT-3 in 2020, reaching an unprecedented 175 billion parameters. The training phase exploited a known database called Common Crawl-a 45 TB dataset composed of raw compressed plaintext [8].

However, despite the enormous dimension, the model showed significant limitations such as weak logical reasoning and poor performance in mathematical problems. What the model tried to do was mimicking intelligence, giving sounding output, but still outputting bad results. This was described in the paper written by DeepMind "Training Compute-Optimal Large Language Models". The main takeaway of this paper is that the large language models at that time were significantly under-trained as a consequence of the focus of scaling.

The empirical behavior of these massive systems was formalized by Kaplan et al. and later refined by Hoffmann et al. through the formulation of LLM Scaling Laws [19]. These laws demonstrated that cross-entropy loss $\mathcal{L}$ decreases predictably as a power-law function of three core variables: the number of model parameters N, the size of the dataset in tokens D, and the total computational budget C, expressed as:

$$\mathcal{L}(N, D) \approx \left(\frac{N_c}{N}\right)^{\alpha_N} + \left(\frac{D_c}{D}\right)^{\alpha_D} \tag{1.5}$$

As these models scaled past certain compute thresholds, they began showing "emergent abilities"-capabilities like multi-step logical reasoning, code execution, and programmatic translation. The most notable point is that these skills were absent in smaller variants and weren't a result of a specific design to perform such tasks. The models ceased to be simple semantic aggregators and began functioning as generalized execution engines.

1.4.6 Alignment, Instruct Models, and the Contemporary State

While raw scale produced profound cognitive capabilities, early base foundation models remained difficult to control. Because they were optimized solely to predict the next token from internet data, they frequently generated toxic, repetitive, unhelpful, or structurally dangerous outputs. These criticalities were due to causes that involved both the training-like the incapability to distinguish minor and major errors or high or low quality data-and evaluation-strictly due to poor metrics used that doesn't capture the value of the generated response [39].

The industry addressed this critical bottleneck through alignment techniques, most notably Reinforcement Learning from Human Feedback (RLHF)-which guides the LLM training by using a reward signal trained on human annotated data-and Direct Preference Optimization (DPO)-which uses the LLM itself as "reward model" on the human annotated data while in the training phase [36].

By collecting human preference data and training a secondary reward model, developers successfully constrained the LLM's token distribution to favor outputs that were helpful, honest, and harmless. The use of these types of techniques resulted in models like InstructGPT, which was able to generate more truthful and less toxic output compared to GPT-3, even considering the lower scale of parameters used by the model-approximately being 100 times smaller [34]. This alignment era transformed raw base models into conversational, instruction-following engines, directly catalyzing the widespread release of consumer-facing tools like ChatGPT, Google Gemini, and Anthropic Claude.

Parallel to closed-source API ecosystems, a robust open-weights movement emerged, democratizing access to hyper-scale models. Examples like Meta's LLaMA series and Mistral AI proved that optimization of data quality, architectural refinements (such as Grouped-Query Attention),

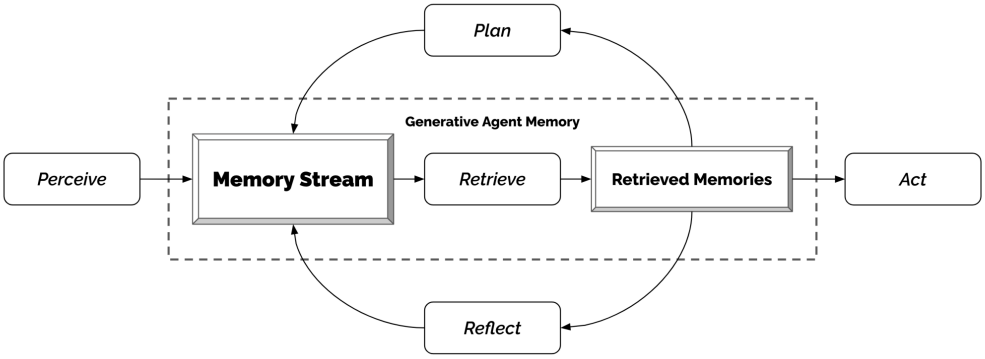


Figure 1.5: Agentic architecture structure example [35]

and extensive instruction tuning allowed smaller models (e.g., 8 billion to 70 billion parameters) to match or exceed the capabilities of early proprietary giants.

In contemporary software engineering, this historical trajectory has yielded a dual-landscape: developers alternate between orchestrating multi-billion parameter proprietary cloud models for highly complex, creative reasoning, and deploying fine-tuned, localized open-weight architectures directly within private, low-latency build environments.

1.4.7 Agentic architecture: an enhanced LLM working strategy

While experimenting with LLM workflows to improve the efficiency and proficiency of the models a new approach that takes reference from the human psychology has been created, the so-called Agentic architecture.

This structure's objective is to regulate the behavior of "agents" working within an LLM's system. To do so it autonomously performs tasks on behalf of a user or another system.

In order to achieve the objective the architecture works by planning, action, maintaining the memory of the previous actions and reflect on the outcomes of said actions. To do so additional components-such as memory structures and additional tools-are present while performing the task.

These components are defined by four concepts that describe the behavior of this architecture and act as base for the base block of the architecture-the agent. The concepts are Intentionality-the ability to decompose a task-, forethought-the capability of planning and anticipating before acting-, self-reactiveness-the ability to monitor and motivate its action-and self-reflectiveness-the capability of "reason" on the process and make corrections if needed for future cases. Each of these are separate steps that are part of the process that works to tackle to the request-which doesn't necessarily need to be made by human-and as time passes it helps to adapt to the user preferences.

The architecture can have different types of structure, which depend on the kind of task it needs to do.

The simplest type uses just one single agent, hence the name "Single-agent architecture". In this configuration the agent works independently on solving the task, to make decision to reach the goal.

Its key strength point is its autonomy, which makes the system simple to design and implement, predictable, fast and cheap. However this composition not only hinders the scalability of the system, making it a bottleneck handling big and complex tasks, but also very rigid and narrow, limiting its action to single specific functions or domains.

This type of architecture shines in the context of applications like chatbots and recommendation systems-where each request is organized and straightforward, not needing the presence of

multiple agents with specialized domains for each task.

Alternatively multi-agent architectures improve the capabilities of this structure bringing the benefit of specialization of each agent, which collaborates with each other.

Under the umbrella of the term multi-agent there are multiple combination of structures. Different structures have different organization and specialization for each agent.

The first type is the vertical structure, in which an hierarchy is present and roles are well defined. In this organization agent report to the leader, which oversees the agents and centralize the control. The strength of this structure is the efficiency of sequential workflows tasks and in the decision process, in which the leader is the only point that makes a decision.

It's this specific aspect that define the weakness of this approach: by having one point of failure not only it can be a bottleneck but it can also become a vulnerability to any issue.

The best use case for this architecture are multi-step linear works like workflow automation and document generation.

The opposite alternative to the vertical approach is the horizontal architecture. This method is formed by a group of model that act as peers in a decentralized system, sharing resources and ideas. To solve a task collaboration is performed while deciding as a group what is the best course of action.

This structure enables innovation throughout the process and allows to process multiple tasks simultaneously. However the presence of multiple peers and the need for making collaborated work can hinder the process in case of mismanagement and slow down the decision process.

This architectures shine in brainstorming, complex problem solving tasks which require different specializations.

A middle ground between the two is represented by Hybrid AI architecture which combines leadership with collaborative flexibility, shifting the leadership based on the requirements.

Allowing this kind of structures allow a versatile and adaptable system, combining the strengths of both of the previous models. However this versatility comes at the cost of a complex system that requires robust mechanism for changing leadership on top of being more demanding in terms of hardware required to run said architecture.

It specializes in versatile, dynamic process like strategic planning or team projects, in which multiple domains must be taken into consideration [20].

1.5 Prompt engineering: shaping the LLM's work

When utilizing LLM's the most important aspect to be evaluated from a user's perspective is how the request is expressed, meaning how the prompt is written. By Prompt Engineering (PE) it is referred the approach of creating the prompt to guide the result generation to be compliant with the requests. When writing a prompt there are several elements that need to be studied in order to evaluate what is the optimal way to express the request:

- what is the context of the request? What is the situation of the problem? What are the boundaries or the domain-specific knowledge needed in the circumstances of the request?
- from which perspective is the request formulated? What information from the context shall be avoided or used?
- how is the result's structure? Considering the request's context what information should be requested and what request should be ignored? What is the desired outcome?

In the context of creation of the prompt there are several techniques used to decide how a task is described.

1.5.1 Core Prompting Taxonomies and Methodologies

Prompt engineering techniques can be classified into several paradigms, each possessing unique trade-offs regarding computational overhead, context window utilization, and reasoning fidelity.

System Prompts and Meta-Prompting

Modern conversational and code-generation architectures expose a distinct configuration layer known as the system prompt. While standard user prompts are designed to dictate immediate, task-specific instructions, the system prompt establishes the broad operational constraints, behavioral boundaries, and safety guardrails of the system. Crucially, this layer delineates the model's persona- which encapsulates the specific domain expertise-stylistic tone, and cognitive boundaries the LLM must adopt throughout a given execution lifecycle.

However, manually engineering precise, robust system prompts remains highly challenging due to the inherent sensitivity of neural networks to slight variations in natural language instructions. To address this limitation, automated prompt optimization strategies have emerged, one of these being the meta-prompting paradigm, which exploits the LLM itself to construct, evaluate, and refine its own prompt templates. Rather than forcing human engineers to manually author instructions through a process of trial and error, meta-prompting utilizes a structured, conversational orchestration loop.

In this framework, the model is initially initialized to act as an expert prompt engineer, dynamically requesting the user or environment to extract functional requirements and constraints. Upon gathering these parameters, the model generates an optimized, example-agnostic structural template that establishes the final system prompt. This self-referential methodology scales efficiently across uniform task categories. This fact is due to the fact that the generated template can be deployed repeatedly without any additional manual tuning.

Furthermore, by replacing human intuition with the model's internal structural understanding of text syntax, meta-prompting substantially improves code and content generation quality. In automated software engineering pipelines, for example, a meta-prompting framework can automatically analyze compiler errors or failed test cases within a system. Using such strategies allows the model to recursively rewrite its own system instructions, isolating and mitigating newly discovered algorithmic edge cases [47].

Zero-Shot and Few-Shot Prompting

These methodologies are collectively categorized under the paradigm of in-context learning (ICL). Popularized primarily by the advent of the GPT-3 architecture-which demonstrated the emergent properties of large-scale autoregressive models without requiring gradient copies-ICL serves as a non-parametric alternative to traditional weight-updating fine-tuning strategies. Instead of modifying the underlying parameters of the neural network, ICL leverages the frozen model's pre-trained distribution by utilizing the immediate prompt context to guide the LLM inference behavior. Within this paradigm, Zero-Shot Prompting represents the simplest approach, presenting the LLM with an isolated task description without any programmatic example or explicit input-output pairs.

While highly efficient regarding context window utilization and token consumption, zero-shot prompting frequently underperforms or completely fails when tasked with evaluating highly specialized or bespoke domains. In game software engineering, for instance, a zero-shot approach lacks the prerequisite domain knowledge to generate functional code for proprietary software frameworks, heavily modified forks of commercial platforms like Unreal Engine, or internally developed, closed-source engines.

To mitigate these constraints, Few-Shot Prompting embeds a finite sequence of input-output exemplars pairs directly inserted in the prompt context prior to the target query. By presenting

these structural pairs, the model dynamically infers the formatting, syntactic requirements, and semantic expectations of the output through latent pattern matching, avoiding the need for gradient-based optimization. In the domain of game development, this mechanism is indispensable for enforcing strict architectural patterns and engine-specific constraints. It allows developers to guide the LLM to strictly adhere to highly decoupled design patterns, or to conform to the precise syntactic and API requirements of distinct scripting runtimes, such as Unity's C# Mono pipeline or Godot's GDScript environment.

Chain-of-Thought (CoT) and Program-Aided Language Models

Complex software engineering tasks, such as state-machine verification or multi-threaded asset streaming algorithms, inherently possess high structural density and intricate logical dependencies. When processed via a standard single autoregressive pass, these tasks frequently suffer from cascading failures, where a minor early semantic error propagates through the generation sequence, rendering the final codebase broken or logically flawed. To mitigate this structural fragility, Chain-of-Thought (CoT) prompting was introduced to enhance sequential reasoning in LLM applications. Originally engineered to tackle the computational bottlenecks associated with direct answer generation in complex reasoning domains, CoT forces the network to explicitly decompose a multi-layered problem into intermediate, linear steps before the generation of the final code output. Mathematically, this shifts the model from direct execution to a joint optimization problem, decomposing the target probability computation into sequential sub-problems:

$$P(\text{Code} \mid \text{Prompt}) = \sum_{\text{Steps}} P(\text{Code} \mid \text{Steps}, \text{Prompt}) P(\text{Steps} \mid \text{Prompt}) \tag{1.6}$$

By establishing this formal, step-by-step reasoning approach within the autoregressive context, the model maximizes programmatic coherence and significantly minimizes structural or syntactic hallucinations [45].

This paradigm is further extended by Program-Aided Language (PAL) models, which address the inherent limitations of relying purely on natural language for deterministic intermediate calculations. Instead of generating intermediate text steps which are susceptible to logical drift or arithmetic errors, the framework instructs the model to generate intermediate, executable code snippets-such as Python scripts-and offloads their execution to an external runtime interpreter or compiler. The deterministic output of this execution is then captured and dynamically injected back into the model's prompt context to guide the final generation phase.

The main difference of this approach from standard CoT lies in the role of the language model; rather than acting as both the planner and the calculator, the model serves strictly as a high-level symbolic translator and orchestrator, relying on deterministic external runtimes for exact computations. Consequently, this methodology is shown to be highly effective for deterministic tasks that challenge standard neural network architectures, such as computing geometric vectors, calculating precise bounding boxes, or generating algorithmic parameters for procedural asset generation [13].

Retrieval augmented generation

While consumer-level Large Language Models (LLMs) demonstrate sufficient proficiency in handling generalized, low-complexity tasks without domain-specific configurations, their performance degrades significantly when applied to specialized domains. As task complexity increases, these models frequently suffer from temporal degradation due to outdated training data or introduce factual inaccuracies, a phenomenon commonly termed hallucination. To mitigate these fidelity issues and increase output reliability, Retrieval-Augmented Generation (RAG) has been formalized as a architectural paradigm. This approach extends the capabilities of a frozen generative model by coupling it with an external information retrieval component, effectively

bridging the gap between parametric memory (knowledge stored in the model's weights) and non-parametric memory (external databases).

The operational pipeline of a RAG system begins when a user query is issued, triggering an initial request to a vector database or document repository for contextually relevant source texts. However, because the raw volume of the retrieved documents frequently exceeds the context window limitations of the generator, the source material must be processed by a "chunker" before inserting it in the context. This component partitions expansive documents into smaller, discrete, and semantically coherent units. This division ensures that target information can be embedded directly into the model's prompt without exceeding token limits or causing the prompt to have excessive information.

Within this architecture, the chunk size functions as a critical hyperparameter-a structural configuration variable determined before the deployment rather than learned during model training-that demands rigorous evaluation and tuning. If the chunk size is configured too small, essential linguistic context is severed. On the other hand, if it is too large, irrelevant noise is introduced, degrading retrieval precision. Once the optimal text chunks are isolated, they are structurally concatenated to the original user query as an explicit context block and fed into the language model. By anchoring the generation process in verifiable, external evidence, this methodology yields more factually accurate, contextually precise, and domain-specific outputs, ultimately allowing the model to perform highly complex, knowledge-intense tasks [37].

1.6 Game development

Throughout all the possible applications of the SDLC the Game development lifecycle (GDLC) is a clear example of how the engineering process and storytelling aspects can be mixed to create an interactive experience.

The products of this workflow can span through various genres and different categories of way it can be played. While the first games provided linear experiences focusing on simple mechanics and simple storytelling, if present, the current state of gaming experience provide complex experiences that combine different domain of entertainment-music, storytelling and graphics for example. To mix all these aspects together in the context of a Software engineering task a specific workflow is needed [6].

1.6.1 Pre-production phase

Before evaluating the implementation an important objective is the focus on "what to make". This can be studied through market analysis and studies on particular aspects that conventional software application don't usually define like user entertainment and enjoyment. Given the standardization of the process this first step requires a greater amount of time, resembling a Waterfall model approach. This similarity is shared in how the needs of the team are handled. Given the high competitiveness of the field and the various skill sets needed in the production phase a clear focus on budget, timing and output control is needed. Even if this could indicate the use of the Waterfall methodology as prevalent SDLC other methodologies such as the Agile are used to address the need of continuous bug fix, feature addition and change from the feedback. Once the workflow framework is defined the first step of this phase is defining the kind of experience through requirements specifications, descriptions of the system and game design documents. While making evaluations on the requirements of the software implementation a strong focus is centered on the non-functional aspect of the product-the entertainment, immersion, game play quality-rather than functionalities. During the redaction of these specifications an important influence is given by aspects external to the SDLC, like communication with shareholders with diverse background.

From this point the focus is then shifted to the technical aspects of the implementation. During this stage the concept of Game Design Document (GDD) is introduced. The central idea of this approach is not only design of the single element present in the project, but also integrating relationships with other components. All these aspect must have a shared vocabulary for increasing the efficiency of the process.

Once the team has a clearer idea of how the game mechanics work an important step is needed before entering production, which is game prototyping. At this point an early version that embodies the basic features of the final product is created so that it can be evaluated early on. This is made possible through the use of platforms such as Twine or Deig [12]. These two platforms allow the abstraction of the core mechanics by using different approaches. The first focuses on a "coding-focused" approach, giving great flexibility in the result at the expense of giving a greater difficulty to users unexperienced in coding practices. The second is based on a block building methodology, at the cost of limiting the genre to a more specific type of narrative games. The benefit of this tool is the drastic lowering of the skills needed to make a project.

While this procedure is carried on the choice of the technical means is an important step. In this step the evaluation of the risks is key given the high cost of the process.

1.6.2 Production stage

Once the product creation workflow is planned the focus is shifted on the implementation of the plan. While the pre-production is centered on the general idea of the game-like the general event witnessed of the player, focusing on the general challenges that the user should be addressing

and what the scope and theme of the game is-this stage emphasize the specific moments inside the game experience.

To address this the storyboard design is made. While approaching this the objective is to formalize the scenarios of the experience. In this stage several concepts must be organized. These elements include elements not directly tied with the code implementation like the story, the character involved in it and the enemies with a concrete AI workflow. This creates a pretty clear view of what a player should know about what is happening, who is involved and what the flow of the scenario is heading. This process can be done on physical boards-to have a better idea of the full picture-or through the use of digital tools to allow an easier access.

Once there is a pretty coherent and precise plan for a scenario the next need is the asset creation. This is the artistic part of the process, in which every component of the game are created. This includes a huge array of skill sets and artistic context like writing, music creation, 3D-model creation and animation, character design and many other depending on the type of video-game that is aimed to be created. The focus of this step is not only to create the components that are needed to play the story, but also to improve the immersion of the player and to translate the view of the creator/s in the most natural way possible.

Given the structure of the game scenario and the assets needed to create the whole environment the focal point shifts to what makes this bundle of materials a video-game, the code implementation. This aspect is vital not only to make everything work, but also to ensure that everything works as smoothly as possible. This side of the process is the more technical part, in which the way the needs are translated depends mainly on two aspects: the programming language implied and the platform that will deploy the ending software.

The first aspect defines the directives of how everything works, being a translation to a language the computer can "understand". The myriad of possible languages can be used extend the possibilities of how each game can be built. The choice of a language should reflect on the context of the application, improving not only the performance of the code execution but also improving the capability of the programmer to write comprehensive code as simply as possible.

Given the nature of video-games in which each scene is made of so called "game objects" the attention is shifted towards object-oriented languages. This type of programming paradigm differs from traditional programming-which focus on defining the flow of the whole program, mainly represented by languages like C-and works by defining what are called Classes. A class is a container which encloses a set of variables and functions, allowing create an object with specific behaviors. This gives the programmer the capability to separate different mechanics throughout different classes and expand in a more structured way. Depending on the architectures and the other programming element of the game the languages that are mainly used are C++ or C#.

While implementing the software the device that uses said program plays a key role in the whole equation. Given a specific platform the programmer is dependent not only on the hardware capabilities-such as raw computing power and battery capacity while considering mobile devices-but also on software limitations. This facet can be divided in a great amount of elements.

A big first factor is the operating system. While the main option until now has been Windows the use of consoles with proprietary operating systems and the recent increasing use of Linux distributions gives a big variety to possible platforms used.

Another element that should be valued is the driver compatibility between the hardware and the software. Drivers are pieces of software that handle the communications between the operating system and the hardware. In the context of video-games this has become increasingly more important. This is not only due to simply running the game, but also for using an increasing amount of technology dependent on the manufacturer's technologies.

Failing to handle compatibility with some hardware can lead to commercial backlash-like how it happened in the recent case of the game "Crimson Desert".

To facilitate the creation of these product there is a specific kind of software, called a Game

Engine (GE). These programs incorporate pre-made libraries especially made for developing video-games, dramatically simplifying the workflow of game development-leaving the focus for coding the main mechanics. There are multiple GE that can be chosen, some being available for the general public-like Unreal engine, Unity engine or Godot- and some proprietary to specific development houses.

Each GE has their own field of expertise and their advantages [30]. For example Unreal Engine-one of the most famous GE- allows to script the behavior of each game object through a block building approach, giving the possibility of making games without any coding knowledge. This unique way of making scripts is an interesting approach to counter the difficulties due to its base on C++ scripting, which might make difficult for beginners-especially with low programming skills. Another interesting example is given by Godot-an open-source GE. Even if its nature as software handled by a small team makes it lose on professional level and in speed of update-especially in 3D aspects-it has its own advantages, like its own programming language made to lower the learning curve for programming significantly.

Depending on the platform that it's desired to program on a specific GE might be used. Some might be capable of programming on every platform-like Unreal engine and Unity-while some might be limited to specific platforms-like Godot, limiting to mobile and PCs. And depending from the GE chosen one or more programming languages is used, giving a possible direction of the skills needed for development.

1.6.3 Unity engine: the industry SOTA

Released in 2005 through the collaboration of few programmers-Nicholas Francis, Joachim Ante and David Helgason [16]. Starting as a Mac OS X exclusive engine, it then gradually expanded its support to other operating systems-both for programming and running the resulting game. Thanks to that expansion, the lack of competitors in some contexts such as web browsers games-which was mainly represented by Adobe's Flash-and the continuous implementation of technologies made this engine incredibly popular. Its initial focus was on being mainly a 3D GE, but it also now supports 2D game creation out of the box. The core strength of this GE is the strong presence of the community given its relatively long history and some particularly useful functionalities such as the Asset Store-a built-in interface that allowed developers to buy or sell assets, reducing the need of wandering from the IDE.

The Unity engine's development interface is characterized by a main workspace in which all operations of game development can be done, the Unity Editor. The view of the editor is subdivided in different windows that do different jobs.

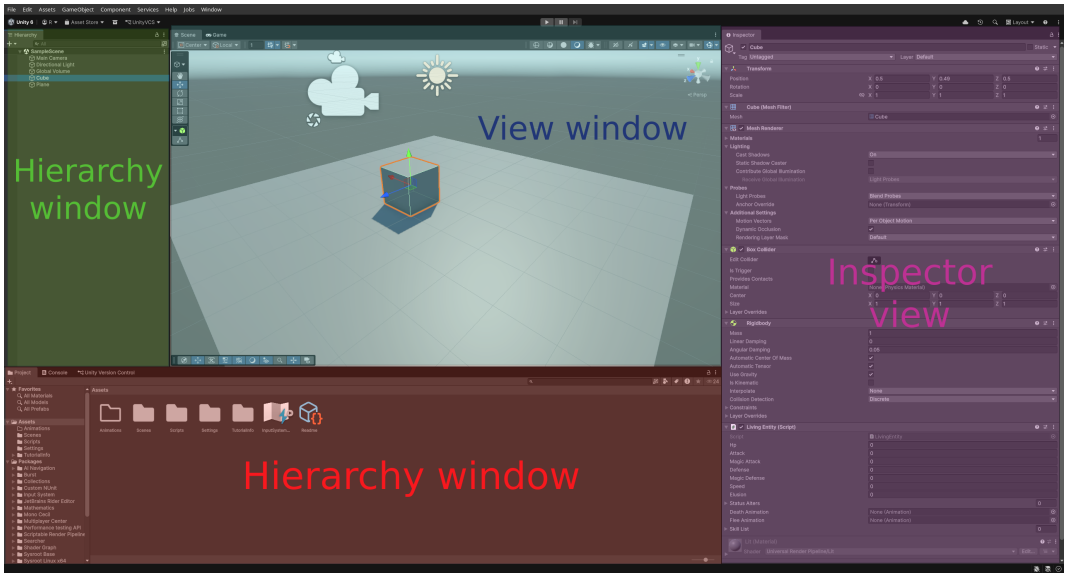


Figure 1.6: The unity editor window.

The Scene view

A game is subdivided in scenes-the collection of all the specific game object needed in a game scenario. A proper division can allow an efficient use of assets and hardware computing power. The scene view is a preview of the world the game objects are in. Inside the view the game object can be selected and manipulated in space. This preview can be explored either by using the arrow keys, the view tool or by using the so-called "flythrough mode"- allowing navigation by holding the right click and moving with the WASD key bindings for movement, EQ key binding for ascending or descending and the mouse for shifting the view, enabling a "game like" exploration. Game object present in it can also be duplicated, copied, pasted and deleted. From this view it is possible to edit aspects like aspects like the scene camera-which edit how the view camera works in the preview-or the management of the granularity of the tools.

The Hierarchy window

The hierarchy window is in the top left corner of the editor and contains all the existing objects of the scene. In it all the object management operations present in the scene view is possible with additional features. Some of these features include making brand new game objects, handling order, visibility and hierarchies between objects. This window is the fundamental part for organizing the scene in a way that makes easier to do modifications and understand the structure of the scenario.

The Inspector window

Located on the right section of the editor, it displays the properties of the selected object which can be modified. These properties are grouped in components, which are libraries of software that handle specific aspects of an object. Other than the one present by default-like for example the transform component that handles position and rotation of the object-other can be added through scripting. These components are handled using the C# language. Any variable that is set as public-making it accessible to external programs-is visible inside the window and its default value can be edited.

The Project window

On the bottom part of the editor this window displays the files and folders present in the project. From this view all the assets can be drag and dropped to any part of the editor to add them if possible. The type of asset that can be managed in this window can also be some type of game object setup that can be used multiple times-called a prefab.

1.6.4 Post-production stage

After the product is created the last phase of the process is test the functionalities of the code implementation and the quality of the cumulative end result. This can be done in two main ways.

To test if the code works properly generally a testing suite is used. It is composed of a series of calls to the functions of each executable, making sure that the implementation work as intended. There are two ways a test suite can evaluate a code implementation: white box or black box. The difference between the two is mainly how the function is seen. While in a black box approach the function internal implementation is unknown-the only thing known is the expected output given a specific input-in white box the whole details of the code is visible. Using one way or another doesn't change what should be checked-which is the result-but the second case allows to check the code in a more specific view. Other than the pass rate-which is the ratio of test passed by the code-other variables like the coverage-which defines how many lines of codes are actually tested. This not only allows to test the actual functionality of the single function but also the relations with the other components of the code base.

However in the context of game development-or in general of highly complex programs-another way of checking the correct behavior of the code is through active experience of the result ahead of the release. This practice is called beta testing and involves different types of individuals: the experts-which control that everything works as planned- and the testers-which act as player that are part of the target audience and evaluate the entertainment value of the experience. This testing phase can happen at any moment during the cycle of the GDLC and can be a valuable feedback source, not only to check that everything works as intended but to have an idea if the game is interesting to the general public that should buy the game.

The last step-often concurrent with the testing phases-is the marketing campaign. This is arguably the most expensive and important step, since it can give the right spotlight to the product that is been made.

1.6.5 The integration of AI in the process

Given how complex the nature of the engineering process actually is the capabilities of AI gave a great boost to the process.

Even considering the recent jump in popularity of AI models after the publication of OpenAI's ChatGPT the use of these technologies was already a big part of the industry, encompassing different parts of the process [29]. These aspects are not only limited to improve the possibilities given to developers, but also working actively while the game is still running. During the process of development these technologies allow to manage how the resources are handled.

Implementation like these allow to create environments that are more natural with reactive NPCs-non playable characters, which are part of the game without being directly controlled by the player.

A pretty interesting case of how this techniques have been used is given by the game "Middle-Earth: Shadow of Mordor". This uniqueness is given by a proprietary AI handler called "Nemesis System". Thanks to this system enemies that previously fought the player would remember the outcome of that fight, recalling it in a future encounter. This not only allowed for a better immersive experience-allowing the enemy to "remember" the player and change its behavior-but

also made possible to dynamically change game difficulty-an enemy that previously defeated the player or have been spared will remember and adapt to the strategy.

Another possibility given is the capability of creating game assets-both in a procedural manner using pre-existing resources and in a generative manner.

Some notable case of the use of this technology can be "No Man's Sky" or "Minecraft"'s world procedural generation-allowing to create brand new worlds to explore for each session. Meanwhile cases like "Clear obscure:Expedition 33" enlightened the possibility of creating temporary assets during pre-production through the use of generative AI, allowing to focus mainly on the implementation of the ideas and leaving the creative lead under less load.

With the increased use of AI models in the context of code generation a lot of the steps involved in the creation of the implementation of the game core loop will be accelerated.

1.6.6 Prompt Engineering in Game Development

The intersection of prompt engineering and game development introduces highly specialized workflows that depart significantly from web or mobile enterprise software development.

Gameplay Scripting and Engine-Specific Generation

Game architectures are fundamentally dynamic, relying on real-time update loops (e.g., Unity's `Update()` or Unreal's `Tick()`). Prompt engineering must be meticulously tailored to avoid generating code that introduces performance bottlenecks.

```
{
  "role": "system",
  "instructions": "You are an expert gameplay programmer
     specializing in Unity C# optimization. Generate clean, modular
     scripts adhering strictly to high-performance guidelines.",
  "constraints": [
    "Never invoke GameObject.Find() or GetComponent() within
       Update() or FixedUpdate().",
    "Cache all component references inside Awake() or Start().",
    "Avoid structural heap allocations (such as instantiating
       objects or generating garbage via lambda expressions) within
       the gameplay loop.",
    "Utilize object pooling concepts where appropriate."
  ],
  "output_format": "Provide clean code blocks wrapped in
     appropriate markdown language identifiers, accompanied by a
     brief technical justification of optimization trade-offs."
}
```

Listing 1.1: Example of an structured system prompt optimized for high-performance Unity C# scripting.

As illustrated in 1.1, structural constraints within the prompt reduce the occurrence of undesirable model behaviors, preventing typical novice programming pitfalls that could degrade the experience performance-wise.

Procedural Content Generation via Prompting (PCG-AI)

Procedural Content Generation (PCG) traditionally relies on deterministic algorithms to build new content based on randomness rather than human work.

Prompt engineering introduces a new domain: semantic PCG. By prompting multi-modal or text-generation models with structural constraints, developers can rapidly generate diverse game worlds. The prompt acts as a boundary box, defining the semantic rules, thematic alignment, and functional syntax that the generative output must satisfy.

The type of content created can span between highly diverse contexts. It can include for example entire structures of levels, or the possibility to extend from the singular concept of 2D levels to 3D terrains-which refers to the topography of virtual environments.

Alternative scenarios in which this technology can be used is in the sphere of the artistic and visuals assets. This can include not only directly visible elements in the game scenario, but also elements that can improve the overall experience of the game. Such elements can be for example the sprites, graphic elements used to represent an action by a series of frames. The presence of these assets is fundamental to perceive the movements and the actions of a 2D object inside the game context. When shifting to the 3D context this topic is also present as motion assets-which are represented by element that are time-based visual contents or can allow the creation of such. These kinds of assets can span from 3D animations for skeletal mesh-a 3D model component that is composed of a hierarchical structure of bones, allowing the movement of the 3D model in a similar manner to a mannequin- to character's faces animations.

While the elements that can be evaluated may include visual architectures purely functional to the game loop context, this evaluation can shift to other concepts that are not directly tied to how the game works but rather on how the game "feels".

Some of the categories of assets that can be thought are the narrative elements. Under this category there are multiple types of resources. One of these is the creation of characters inside the context of the game controlled by mechanisms handled by generative AI, allowing for creating an higher level of immersion not only in the context of living the story but also how the story evolves through time given the actions of the player.

Additionally it can also include aspects like sound effects and music, that can greatly enhance the player's capability to feel as part of the story's environment.

In order to face this process most research on the topic tend to revolve around the use of Generative Adversarial Network (GAN) based approaches. This type of use of generative AI involves a the network that has a loss function and multiple agents which compete in order to reduce said loss function. This allows to integrate conditional inputs and attention mechanisms, enabling a better creation of assets considering the features required in the context of the game scenario [24].

Refactoring, Porting, and Legacy Code Maintenance

The game industry is notorious for its reliance on legacy codebases and proprietary scripting tools. When studios transition from older engine iterations to modern frameworks, prompt engineering provides a scalable automation methodology.

Developers can construct complex translation prompts that map legacy memory management models, custom libraries, and obsolete rendering abstractions to modern, secure equivalents.

To achieve such objective diverse methodologies can be used, which rely on LLM trained on vast codebases that have multiple programming languages.

Where some possible approaches might rely on techniques already mentioned previously like Reinforcement learning algorithms, there are multiple methodology that might be used in this context.

One method is represented by the Lexical pattern matching. In this approach the code is treated as a sequence of tokens, then analyzed and manipulated trying to spot patterns of "code smells"-features that indicate generally defects in programming. In case the model spot some patterns it can either relocate methods, break down functions in smaller snippets or rename ambigous variables.

An interesting alternative path is brought by modeling the codebase to be refactored as a tree structure, rather than directly analyzing it as text. Once the tree structure is created it is processed and encoded into an embedding so that a neural network can process it and manipulate it to result in a new code state that must be able to compile and retain the original logic of the entire codebase.

One approach is the Abstract syntax tree (AST). This structure captures the hierarchy of the code by evaluating mainly structure and syntax.

Another methodology is represented by the lossless semantic trees (LSTs). Unlike the previous procedure this tree structure preserve style formatting and include type information for each code element, allowing safer semantic code searches and dependency upgrades/migrations.

The use of LLM-based approaches into the refactoring process are multiple. One of the first is the capability of improving the quality of the final result of the process. Given the error-prone nature of the process this approach can decrease the possibility of introducing new bugs during the code translation phase. Additionally employing this tools can remove some of the load off the developer's shoulders by doing the repetitive refactoring tasks. Having less work needed the developer can focus on the more complex part of the refactoring process. The possibilities given by the integration with most integrated development environments (IDE)-programs used specifically for programming having both text management and compiling capabilities-decrease the friction with the workflow. Given also the massive scale this mechanisms can have it can accelerate the timings of the refactoring, which can usually take tremendous amounts of time in the context of manual refactoring.

While these methods can improve drastically the quality of the work and can save time for the developer to work on less redundant tasks, a few considerations are required. One factor that can be evaluated is the fact that these technologies excel in low-level, consistency-oriented refactoring. This is due to the fact that AI tools can't be an alternative to a human developer's expertise and judgment. The fact that a human's perspective is still needed means that the developer must control and evaluate each recommendations the AI system makes, as they often lack the context awareness of complex structures. This mean that rather than using this tool as a recreation mechanism the scope of the utilization of LLM-based approaches should be reduced to cleanup tasks, letting the developer experiment while keeping the code structure in check. Additionally, the presence of test suites is necessary in the context of the process to make sure that the initial behavior is maintained after the refactoring process [9].

1.6.7 LLM's technical limitations

While Large Language Models show noteworthy talent in generating syntactically correct code implementations, their deployment within production-grade software engineering pipelines is limited by deep architectural limitations.

The most prominent among these is the structural phenomenon of hallucination, wherein a model generates code that is syntactically plausible but logically flawed, non-existent, or insecure. In an autoregressive architecture, tokens are sampled based on probabilistic distributions rather than a deterministic understanding of execution logic or ASTs. As a consequence an LLM may invent non-existent API endpoints, reference deprecated library dependencies, or introduce silent semantic bugs that evade standard static code analysis and require intensive human debugging.

This issue is severely boosted by the constraint of context window saturation. In software codebases, particularly in complex domains like game development, may involve highly non-linear and deeply interconnected systems. A single script implementing a player character controller may depend on global physics configurations, animation state machines, network replication protocols, and specialized memory management pools. Compiling and inserting this massive, multi-file technical context into an LLM's prompt prefix risks exhausting its context window token limit. Even when using modern architectures featuring high context capacities, models frequently suffer from the "lost in the middle" phenomenon, where retrieval fidelity dramatically

degrades for information located in the center of the prompt context, leading to incomplete or mismatched code generations.

Additionally, the integration of LLMs introduces a fundamental friction between stochastic fluidity and deterministic consistency. The foundational premise of software compilation relies on determinism: given identical source code, a compiler must return identical binary outputs. LLMs, by contrast, are fundamentally probabilistic. Even when setting the sampling temperature parameter to an absolute minimum to enforce greedy decoding, subtle variations in context construction, floating-point arithmetic across parallel GPU hardware, or unannounced upstream API updates can cause a prompt to generate drastically different code implementations over time. This lack of strict reproducibility undermines continuous integration and continuous deployment (CI/CD) pipelines, forcing engineering teams to construct elaborate, automated regression and unit-testing layers for the sole purpose of validation of the stability of AI-generated code.

1.6.8 Ethical doubts and challenges

Beyond technical vulnerabilities, the utilization of LLMs in software engineering exposes complex ethical, legal, and social challenges. The first legal risk centers on intellectual property and licensing contamination. Generative foundation models are trained on vast, heterogeneous internet scrapings containing open-source repositories governed by highly diverse licensing structures, ranging from permissive (e.g., MIT, Apache 2.0) to strictly copyleft (e.g., GNU General Public License / GPL). When an LLM generates a code snippet, it may inadvertently reproduce sections of copyleft-protected code without propagating the mandatory license attributions. Integrating these outputs into a commercial, proprietary codebase can inadvertently trigger massive violations, putting software enterprises in the risk of copyright disputes and intellectual property litigations.

From an architectural safety perspective, LLMs may introduce severe security vulnerabilities and algorithmic bias. Because training datasets heavily reflect legacy internet codebases, models frequently insert into the generated result outdated, unoptimized, or explicitly insecure programming paradigms.

If developers start to blindly depend on these generations to increase development speed, the result is the increased proliferation of security vulnerabilities. This risk is intensified by the concept of "automation bias"-the tendency of human operators to over-trust automated systems under. This aspect can indulge junior or hurried developers to bypass rigorous code reviews under the false assumption that structurally clean code is inherently secure.

Finally, the widespread adoption of AI-driven generation presents profound social implications regarding knowledge deskilling and the degradation of engineering agency. As natural language abstraction layers increasingly detach the developer from low-level systems architecture, there is a systemic risk that the incoming generation of software engineers will fail to develop deep mental models of algorithmic complexity, memory management, and hardware optimization. In high-performance environments like game engine programming, where squeezing efficiency out of constrained hardware is paramount, a reliance on superficial, AI-facilitated scripting can bottleneck long-term innovation. The role of the engineer risks being downgraded from a creative, first-principles problem solver to a passive "prompt driver" and code validator, shifting the nature of software craftsmanship entirely.

Chapter 2

System model

2.1 The scope

The scope of this investigation is to evaluate the structural and operational influence of automated code generation methodologies based on LLMs within the SDLC, restricted within the context of video game development. By analyzing the intersection of generative artificial intelligence and interactive software design, this thesis explores how the use of this technologies alter traditional production timelines, system architectures, and engineering workflows.

The primary objective of this study is to measure the efficiency, syntactic fidelity, and architectural viability of LLM systems during the active compilation and implementation phases of game engineering.

To evaluate these dynamics, an LLM-based framework is formalized within a controlled development sandbox to simulate and evaluate specific, multi-layered code generation scenarios. This framework is leveraged to isolate the design stage, which is performed by the developer, and the implementation phase, which is executed by the LLM.

2.2 Related work

While being a rather new environment in which these technologies have been studied, the use of AI in the context of game development has been evaluated by multiple studies.

The main focus, however, has been been specifically to highlight the functionalities of the transformer architectures. The scope has been put on the concept of LLMs as "game producers", specifically to study how these technologies can be used to substitute the process altogether by implying one or more agents into an architecture that can generate a full experience.

2.2.1 MarioGPT: GPT-2 fine-tuned as level generator

One of the first approaches to the utilization of the AI models into game development has been done in the context of PCG.

This has been the case of the MarioGPT study, which focused its efforts into evaluating the impact of generating new levels for the original game of "Super Mario bros".

To do so a fine-tuned version of the GPT-2 model has been used, specifically using the distilled version called DistillGPT-2. This is the result of a technique that allows to transfer the knowledge of a bigger model-in this case GPT-2-into a smaller model. The result is a model that retains similar performances while having a fraction of the dimensions of the original model and reaching higher speed due to its lightweight structure.

To train and fine tune the specific implementations a corpus of 37 levels taken from Super Mario Bros original game and Super Mario Bros: The lost levels-a canceled version of the second game, which featured an higher difficulty level. This corpus was then encoded by converting

each tile into a string representation. To avoid the problem of the encoding of the level's end in the training phase all the levels used were stitched together into one big level.

To allow the model to create new levels two specific mechanisms were used: one to generate the level and one to mutate the level. While the first mechanism is quite straightforward-generating an entire level given the model weights-the second mechanism is what tries to create more diverse results.

To tackle this operation the mechanism employs an algorithm that explores the level and generates a predicted player path. By doing so parallel sections can be found and mutated randomly using another prediction of the LLM model to create a new version of the section.

This architecture allows to create new level either by being by being the result of a prompt-guided generation or completely random output-in which the guiding prompt is randomly chosen-of the level [40].

By employing an architecture similar to the one indicated the user can start creating new levels, but it has little agency on the structure of said levels other than indicating how the general structure should be.

2.2.2 The expansion of the paradigm: GameGPT, an agentic architecture built to make games

While the previous architecture focused strictly on creating new assets in game, another study took this approach and expanded it to the structure of the game.

This was the case of the GameGPT study, which combined the agentic architecture with the intentions of the user even further. This structure is based on a dual-collaborating methodology-in which couples of models collaborate in order to achieve its sub task. By employing this method it is possible to exploit the advantage of having multiple different perspectives without hindering the speed of the decision-making process excessively having less negotiation needed.

Unlike the previous study, which exploited one model that did most of the hard work both to project and generate the requested result, it focused on the utilization of multiple agents, each having one specialization. The structure contained multiple models in its organization, each having its private memory system and the possibility of communicating with the others through a shared discussion. This allows to gradually narrow down the task until completion. These models are distributed in four specific phases, which separate the creation process into a linear and simpler set of sub-tasks.

During the whole process the user can participate in the creation of the result, resulting in an approach called Human-in-the-loop (HITL). This allows not only to reduce hallucinations, but also to create a more focused result.

The two main challenges that this study wanted to address while using AI-powered strategies are the following: hallucination and redundancy-which in this context is bound to boilerplate code or multiple code section doing the same functionality.

The planning phase

The first step of the process, in which the first agent-the so-called "development manager"-generates a task plan from the initial prompt given by the user. This initial generation uses as reference one of multiple game template, one for each game genre that the architecture can handle-the study focused on five genres. This templates define not only the type of game but also a standardized plan, guiding the development manager to finish the task with all relevant information needed.

When the plan is generated an additional model-called the "reviewer"-analyzes and focuses on reviewing to reduce hallucination and redundancy. The results of the review will be sent back to the development manager, which will utilize it to create a more refined and precise work.

Throughout this phase the development manager shares updates on the process to the user. This allows the architecture to receive any kind of suggestion or correction in order to correct the course of the creation as soon as possible.

The Game development task classification phase

The plan that is the result of the previous phase will serve as base for this phase, in which it will be disassembled into a list of task. This specific step requires high accuracy in identification of task type and the arguments in order to create precise and specific tasks.

To do so a combination of a BERT model-which categorize each task-and another model-which is referred as "development engineer" and is tasked to identify the arguments of said task-is utilized to define a final version of the plan to generate the code.

Before proceeding to the final section of the process the reviewer agent is employed to confirm the quality of the retrieved tasks. Additionally the reviewer can interact with the user if the model deems necessary to ensure that all the necessary argument details are present.

Finally the reviewer is also responsible for the evaluation of the necessary dependencies. To encapsulate all the necessary dependencies a directed acyclic graph is used to represent it and then a breadth first search algorithm is used to traverse it. The traversal gives the sequence for task execution, ensuring the order and a systematic way of task execution according to the dependencies

The code generation phase

After receiving the task plan from the previous step the fourth agent-called the "game engine engineer"-generates the code that is needed to solve the task required.

During these process a few methodologies were used to improve the quality of the final result. The core of these methodology is based upon focusing the efforts of the inference to decouple the generated code. This process requires to divide each dependency as much as possible, creating short snippets of code more manageable and more easily correctable.

Given the low dimension of each snippet a in-context learning strategy of few-shots can be utilized to improve the quality of the model inference. To do so multiple examples of snippets are given inside the prompt of the agent, effectively reducing hallucination and redundancy.

To further improve the quality of the end result a candidate selection mechanism is utilized. This involves in creating a set of K code snippets for each task, which are tested and presented to the user, who gives a feedback. This response, along with the testing process, is used to identify the optimal solution among the ones generated.

Additionally to further improve the results an in-house lexicon created with an extensive repository of code snippet-each annotated with labelers that include the intended purpose-is used to fine-tune the model.

The final code resulted from the process is then evaluated by the reviewer agent, which inspects the codebase. This inspection involves a comprehensive assessment, trying to find any discrepancy between the requests and the actual result of the process. The reviewer not only flags those discrepancies but gives recommendations for refining such code. This "correction" is shared with both the game engine engineer and the user, which can provide suggestions if needed. If those suggestions are deemed appropriate by the reviewer they are incorporated in the corrections needed, improving iteratively the code generation approach.

The testing phase

The result of the generation phase is then checked by the last model-called "game engine testing engineer"-which executes the generated tasks keeping the sequence formulated in the

previous phase. The execution process requires sending the code of each task to the game engine, performing the execution while keeping track of the logs.

Once the execution of all the task is done the testing engineer consolidates the log in a brief and comprehensive summary, which is presented to the user. In addition to that any observed traceback are observed and reported, as they indicate adjustments needed to refine execution or code generation.

An additional strategy used to reduce the impact of hallucination and redundancy throughout the whole process involved an additional step in the training for the models present in the planning and code generation phases. During these steps an instruction tuning process-mainly guided by in-house databases containing input/output pairs-was performed. By employing an approach of this fashion the gap between the requests expressed in natural language and the behavior the LLM should performed is partially covered [10].

2.3 The limitations of the approach

While studies similar to the one presented previously show the potential of the approach, giving examples of possible new ways to approach the task of game development, it posses several elements that need to be keenly kept under check.

These factors are not limited only to the context of the final result but consider also the future implications of the use of these methodologies in the scenario of development in general.

Most of these critical aspects stem from the core idea of these studies.

Rather than presenting a specific way to introduce the technologies in the context of development of video games, these studies focus their energy on presenting their LLM-based approach as an alternative to the original, human-reliant approach.

While not being a problem by itself, as it highlights and demonstrates the capabilities of these new technologies showing new possible ways to tackle game developments, its implications might create possible complications down the line.

The first set of the main challenges revolves around the final product of the process.

Relying on a probabilistic infrastructure like an LLM-based, in any possible configuration-both in chat, single agent or in an agentic infrastructure-removes specificity to the final result.

This is caused by the limitations of the context window, effectively limiting the prompt specificity.

For example, by trying to create a game whose prompt describes a turn-based game, which has exploration elements and recruitment mechanics of the enemies encountered throughout the game loop how can an agentic infrastructure discern the difference between a game like "Pokemon" or "Persona 5 Royal"? 2.1



Figure 2.1: A fun experiment: what kind of prompt would be necessary to explain a game in a exhaustive manner?

This element not only requires an excessive time to pin point the game's genre, but also undermines the "style" of how this product presents itself. Components like menus, UI interfaces and formats can become ignored or undervalued factors in the process of game development. This can be caused by the the "distance" between the user-that gets "downgraded" from creator to advisor-and the creation process.

As a consequence of the devalue of the process and the speed it can have during its production time the perceived value of the final result can plummet, not because it is of a lower quality but rather because it feels less "personal" and a more generic result.

But even without mentioning about the ludic and creative aspects of the process, the use of these AI-powered approaches can arbor complications in the technical side of the process. By entrusting most of the work to the agentic architecture the risk expands from the specificity of the single creation to the future capabilities of programmers and engineers to solve these problem by themselves. This risk can be the loss of the "know-how"-literally "knowing how to do something", the knowledge of a process internal steps' action and requirements. While this can become a valid tool for expert programmer that can use for speeding up trivial tasks an over-reliance can be the origin of a stagnation or worsening of the programmer's capabilities in the long run.

Another critical point that should be discussed is how the technologies used can be affected. Relying excessively on automatic approaches can diminish innovation in some case where the lack of core functionalities gave birth to brand-new technologies-like the case of Nolla Games' "Noita", which was based on a custom game engine tailored specifically to simulate fluid physics in a pixel art environment. Given the probabilistic nature of these technologies a risk can be the stagnation of the technologies and the underlying ideas.

While these topics highlight actual risks, it's worth noting that all these elements do not imply that the technology by itself is flawed or damaging for the process. The main critical points stem from its use as a generalist tool and from the approach used to it as an "additional

brain that does whatever the user says". Considering that most of the programmer's approach even before the birth of consumer-level LLM relied on community-driven efforts-by searching the "right" snippet on the internet and pasting it into the implementation-the use of copilots like these can improve the efficiency of the workers but require to separate the thinking process from the operational process.

2.4 Shift in paradigm: from whole experience architect to specific component constructor

In order to avoid the limitations of these paradigms without completely disregarding the potentialities that this technology offers a paradigm shift is needed. The objective is keeping the strengths of the utilization of LLMs while trying to avoid or at least reduce the limitations both in term of technical application and in terms of artistic view. Instead of relying on the AI methods to substitute the engineering process this study focuses on trying to diminish the trivial work needed after the implementation details are decided.

This type of methodology is not something completely unseen, as it has been evaluated in a similar direction in recent studies. One example is the one proposed by Anma Hassan's "Automated Unity Game Template Generation from GDDs via NLP and Multi-Modal LLMs". The focus of the study is to utilize the concept of Game Design Documents (GDD) applied to code generation during the process of game development. GDD are specific documents that describe the whole structure of the game experience that has been imagined, containing concept art, diagrams, text or any applicable media usable to describe meticulously the developer's choices. This kind of document is inserted in a custom tool made specifically to convert the ideas into the script needed to implement such design. The structure is composed of multiple blocks, which follow the following general workflow: extraction of the key element of the tasks, converting those elements into requirements, generate the dependency graph of the implementation and finally generate the code required to be conform to the requirements. The implementation utilizes the FastAPI endpoint to interface the model with the rest of the pipeline. Throughout the implementation of this framework multiple models have been tried, most of them coming from the LLaMA family. To blend the generation of code with the game development in the most immersive way possible, trying to avoid to switch between different interfaces, a custom Unity Editor integration tool has been created which receives in input the GDD-which can be in multiple formats ranging from pdf, txt or docx for example-and shows the whole pipeline work and results directly to the user. To evaluate the result four main dimensions have been evaluated, which analyze the code generated in terms of quality for each individual class generated and its validity in the context of the validity for the design. The first evaluation dimension is the compilation success of the generated code. This works as baseline for the quality of the code created, because it is the initial requirement for testing the codebase. The second point is the adherence to the GDD. While it is important that the code is able to at least execute it is as critical to make sure that the implemented code reflects the mechanisms that the GDD specified. Then it is studied the use of best practices in the context of programming in the Unity environment. This aspect indicates mainly the proper use of the classes, components and functions internal to Unity's system. Finally the last evaluated dimension is the modularity of the code. This aspect values mainly the capability of the models to create manageable classes and methods. By using three main standardized GDDs the framework has been tested on 5 models: LLaMA 3 8B Instruct, Gemma 2 Instruct, Qwen 1.5 Chat, LLaMA 4 Maverick and a fine-tuned model based from LLaMA 3.

To create the fine-tuned model a repository from the "Mix and Jam" Youtube channel was exploited. This archive contains high-quality recreations of popular video games mechanics, implemented in Unity. From each implementation a GDD was generated and used directly in the fine-tuning process. The final results highlight the capabilities of fine-tuned LLMs, on how

creating a model specifically tailored to manage tasks in the context of Unity game development can reach greater heights than any of the tested generalist models. The research explain this superiority as a result of three main contributing factors: Domain specific training-which given the training specialized in GDD-code couples gave the fine-tuned model specific knowledge that gave an advantage-, Structured approach-which improved the result's consistency and relevance against raw text input-and Unity-specific patterns-which were another result of the fine-tuning, contributing to have specific design patterns and best practices.

One important result this research has highlighted is the consideration that understanding the game specifications is not enough to build working code [17]. This last intuition is the base in which the approach of this study bases its work.

The core point that the proposed methodology wants to change is the use of the models. While in the previous paradigm the implementation was completely left up to the architecture to create and in the previous study the work is restricted to manage the structure of the project adhering to the design, in this methodology the LLM receives the specific design of the structure it needs to implement and creates its codebase, without having any additional tasks. While all these methodologies focus on using LLMs in multiple types of tasks the focus is again compressed to one specific task-which is code generation.

2.5 Model description

This section will cover the high level paradigm and methodology that structured the work of this study.

The concepts explained in the next paragraphs will contain not only the main idea behind the methodology used, but also the challenges tackled in the context of the study.

2.5.1 Study approach definition: Generative AI as a Design Translator

This study proposes a fundamental paradigm shift in how generative artificial intelligence is utilized within the software engineering pipeline. Instead of relying on the model to drive architectural intent, the LLM is treated strictly as a deterministic translator of formal design blueprints.

While the exploratory, open-ended approach may suffice for loose web scripts or isolated application prototyping, the use of LLMs may introduce severe architectural vulnerabilities when applied to complex, performance-critical software environments like real-time simulation engines. For this reason the first main step of the approach proposed involves only the developer.

The initial phase requires to define what is the type of product it is desired to implement. After deciding the macro-topics of the final result-like the genre, the target audience and the stylistic choices-the next activity is to try to separate each context by defining its rules and mechanics. Once each scenario has been defined the following stage is to define the boundaries of the scenario and each component in the context. The scope of this part is to start to understand and define the relationships and interactions between all the actors-the game object inside of the same scenario.

This is the core phase that allows to pass from a generic development workflow to the proposed framework, as this is the point in which all the necessary information is present and can be handled to guide the generation of the result class.

All the information that has been gathered it is managed to create the base structure that each game object will be utilizing in the context of the Unity project scenario: a "component". This specific part is a C# script that define a specific behavior inside the game evaluation of the game. Utilizing the information inside the project applied to the specific functionalities a plan specific enough can be formalized to create the prompt needed to generate the required implementation.

The model's role is reduced to a high-speed synthesizer, translating precise, multi-layered natural language definitions and structural constraints into compile-ready, syntactically correct C# source code tailored precisely for the Unity engine. Additionally restricting the scope of the LLM's work will allow to utilize domain-specific models that will be able to improve the quality of the final result.

Under this methodology, the cognitive responsibility of engineering the logic, defining the structural relationships, and constraining the software architecture remains entirely with the human software engineer while the LLM takes care of the practical part of implementing the code already formalized.

2.5.2 Shifting Focus from Code Writing to System Engineering

For decades, a significant portion of a game developer's operational time has been consumed by the mechanical syntax of coding: managing boilerplate code, wiring up event listeners, creating standard variable getters and setters, and maintaining repetitive engine-specific loops. This focus on syntax often creates a cognitive bottleneck, distracting the engineer from the high-level design of system mechanics, performance optimization, and architectural scaling.

By redefining the LLM as a translator rather than an autonomous thinker, the developer's core responsibility undergoes an execution shift.

This change allows the developer a major freedom into the implementation by decreasing the need to write code. Writing code by hand is traditionally the phase where abstract design ideas are turned into source files. In this methodology, that phase is replaced by precise prompt orchestration. Doing so the coding task transforms into compilation. The developer no longer spends hours typing out syntax; instead, they spend that time defining structural blueprints, mapping variable lifetimes, and designing precise execution loops.

The software engineer moves out of the low-level execution loop and steps into the role of a system architect. The main task shifts from figuring out how to write a syntax-valid loop to engineering the underlying rules of the game mechanic itself. The developer works on designing finite state machines, structuring data layouts, and enforcing strict separation of concerns, leaving the mechanical implementation to the model.

It is worth nothing that this specific aspect of the process-the complete substitution of coding with the generation process-is an extreme case that this study wanted to evaluate to experiment with the possibilities of the LLM in a more focused environment. This framework can be used in an hybrid manner along with traditional coding approaches. Doing in this way the process can be focused on coding manually the crucial parts of the implementation, leaving trivial elements to the LLM architecture.

This shift directly addresses a major issue in game development: technical debt caused by loose, ad-hoc programming. When an LLM is given free rein to "invent" code, it often generates disorganized, unoptimized logic that ignores the engine's architectural constraints. However, when the model is used strictly as a translator, it is forced to operate within a rigid sandbox defined by the engineer's prompt structure. The human engineer defines the exact constraints, and the model simply fills in the syntactical details.

2.6 The main challenges

While trying to evaluate the feasibility and quality of an approach of such fashion the presence of some contexts. These contexts will range between three main categories, each describing a core feature of the whole framework.

2.6.1 Mitigating Hallucination and Architectural Drift

The risk encountered when integrating generative AI into production-grade software environments is semantic hallucination.

The design translator paradigm aims to drastically decrease architectural risks by gradually focusing the core implementation from its scenario to the minute details of the implementation. If an LLM is prompted to generate a complex subsystem, such as an autonomous enemy behavior tree, from an unconstrained, high-level natural language prompt, the network must sample tokens broadly across its entire multi-domain training distribution. This excessively vast operational search space often inserts unpredictable, fragmented mixture of incompatible design patterns that can easily conflict with and corrupt the existing codebase.

On the contrary, when the model is utilized strictly as a design translator, its creative search space is intentionally and drastically restricted by the prompt structure. The proposed prompting framework acts as a rigid set of architectural guardrails, effectively filtering out irrelevant paradigm data before token selection occurs. Because each specific element of the intended implementation's layout is precisely predetermined by the human engineer, the prompt structure actively strips away semantic ambiguity.

By providing the model with a complete, deterministic structural blueprint and explicit execution context within the context window, the network is not required to guess the project's layout. The translation process consequently becomes highly predictable and repeatable across distinct generation sessions. The overall probability of hallucination drops dramatically because

the model is no longer tasked with designing the underlying system architecture, meaning its cognitive load is compressed entirely to translating explicit design specifications into valid compilation syntax.

2.6.2 Preserving Architectural Integrity in a Game Engine

A modern game engine represents a highly sensitive, performance-critical computational ecosystem where data layouts, hardware cache optimization, and thread execution schedules are deeply linked. Mixing the architectural responsibilities of scene-dependent game object actors with those of decoupled, data-driven class configurations rapidly degrades runtime performance, introduces frequent cache misses, and compromises long-term project scalability.

By utilizing generative artificial intelligence strictly as a design translator rather than an autonomous architect, the human engineer maintains deterministic control over the engine's underlying structural paradigms.

The developer explicitly defines the rigid logic boundary and class ownership patterns before a single line of code is ever generated by the model. This preventative constraint ensures the large language model is never granted the architectural latitude to inadvertently construct monolithic "God Objects" or introduce unoptimized, redundant data duplication across transient scene entities.

These operational boundaries are not merely restricted to the localized behavior of the single class, but actively incorporate its internal dependencies and structural constraints, which must be respected to ensure successful composition when attached to a scene actor. The presence of pre-existing component dependencies, configured within a specific engine layout, can render the generated code invalid if inter-component relationships are violated. This structural mismatch occurs when generated elements attempt to call specialized methods or access data structures that are improperly bound, resulting in severe component utilization conflicts.

Instead of operating in isolation or relying on general assumptions, the generative model is provided with a highly structured blueprint within the prompt context.

It allows development teams to compress production timelines and engineer complex, highly performance-critical systems by decoupling high-level mechanic design from the manual, error-prone labor of typing syntax. Consequently, as the software matrix scales in size and complexity, the underlying codebase remains syntactically clean, hardware-performant, and under the strict architectural governance of the system engineer.

2.6.3 Integrating the framework into a normal work context

One last condition that must be checked before integrating the framework into a working environment is the part that can be described in the context of non-functional requirements.

The structure produced might even reach quality standards of incredible heights, but its value would be incredibly undervalued if such platform could make the experience "worse" or "slower". Another important problem is the capability of this infrastructure is its capability to be easy enough to try.

This can be tackled through the use of programs and extensions already widely used-especially by exploiting open-source infrastructure and/or the ones that have a big community around it-or present in such a manner that implement the organization is as simple as possible.

This is because the framework can be the best and most precise tool, but if its impracticality overcome its capabilities it will require intense training to be used.

Chapter 3

My main contribution

To establish a logically sound and capable framework for integrating AI within the production of a real-time interactive simulation, it is necessary to formally analyze the architectural building blocks of the development environment. In the context of the Unity engine, software engineering paradigms expand from traditional C# architectures.

The structure and behavior of the actors of any project built on Unity is governed mainly by two type of classes native of the game engine's libraries: Monobehavior and ScriptableObject. The developer must understand its underlying structure and core functionalities in order to create structurally sound and efficient code that utilizes appropriately these classes-especially in the context of utilization of AI-powered development frameworks.

3.1 The MonoBehaviour Class: Component Composition and the Native-Managed Boundary

The MonoBehavior class is the core of the logic implementation from code implementation into Unity's scenes. Its importance is especially evident in the context of Unity's Component-Based Architecture-a paradigm in which software is built from exploiting pre-defined, reusable codebases. In order to apply the framework in the context of developing in the Unity platform, the understanding of its core functionalities is key.

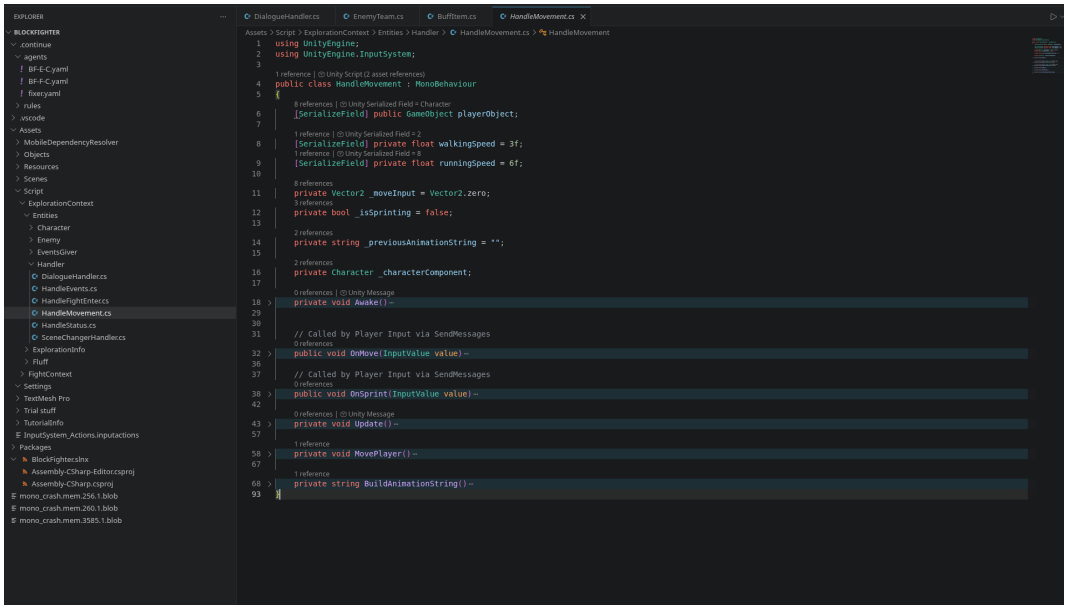


Figure 3.1: An example of script implementing the HandleMovement component for a game object

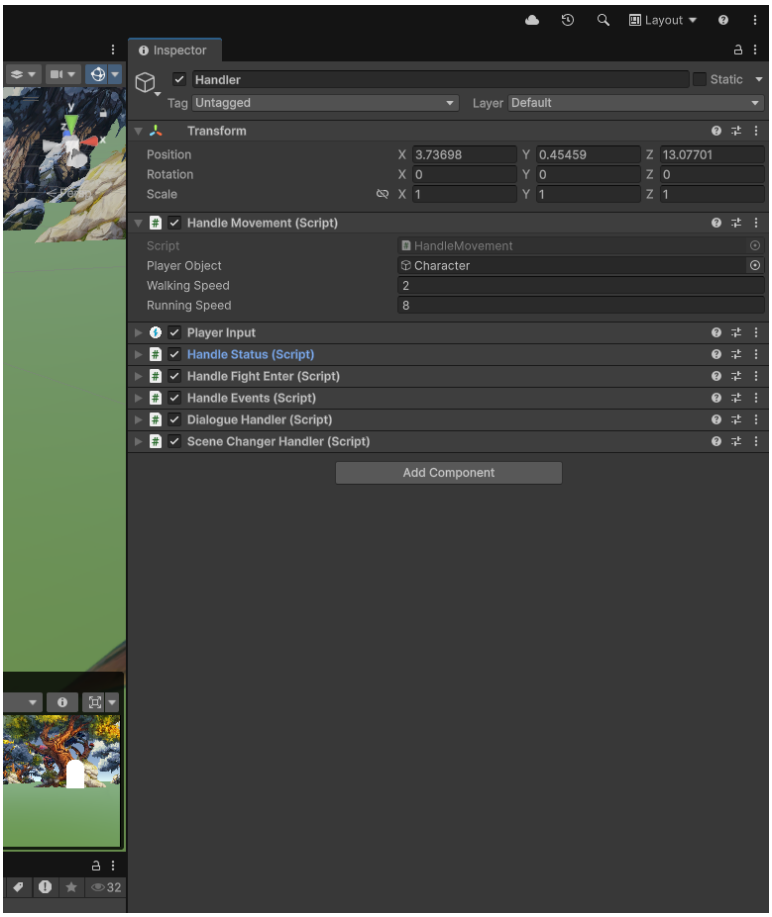


Figure 3.2: The view of the HandleMovement component in the inspector view of the Unity editor window.

A MonoBehaviour cannot exist independently, it must be bound to a GameObject. The connection between the two inherently links this class's lifecycle to a specific entity inside the scene hierarchy. This binding grants the script access to a highly deterministic, frame-dependent lifecycle execution order:

- The Initialization Phase: Governed by Awake() and Start(), this phase sets up internal references and caches external components. Launched as the game object gets instantiated, during this step dependencies like physical sensors, rigidbodies, and navigation mesh agents are bound.
- The Physics Loop: Governed by FixedUpdate(), this loop runs on a discrete, fixed time interval decoupled from the rendering framerate. This stability is critical for physics-based applications, such as steering behaviors, raycast-based vision sensing, and collision avoidance algorithms, ensuring that the system's calculations remain consistent regardless of frame drops.
- The Logic and Frame Loop: Governed by Update(), which executes once per frame. It handles variable, real-time calculations such as checking distance thresholds, updating sensory memory timers, or evaluating transition conditions within a Finite State Machine (FSM).

From an architectural standpoint, the primary responsibility of a MonoBehaviour within the game ecosystem is acting as the agentâs physical interface to apply the desired logic. This factor is a consequence of its tied connection to the game object, allowing the class to have access to the entity's components. Such element may give access to features like spatial information through the Transform component, environmental geometry via physics engine or translating the behavior visually through the use of animation components.

This role is vital in the lifecycle of a game development process, giving the developer the elements to rule the behavior of the single object according to its current situation in the game scenario. When added in the setting of the game context, this aspect allows to decouple the behavior of each single game object from the specific use case. The clear advantage of this modularity is the capability to reuse this components if properly designed [42].

3.2 The ScriptableObject Class: Serialization, Memory Optimization, and Flyweight Data Design

While the MonoBehaviour class excels at managing physical presence and frame-by-frame execution, its disposition as a game object-bound element makes challenging the creation of logic independent from the current scene-like for example currency management. The use of such element becomes highly inefficient when tasked with storing static configuration data or complex behavioral layouts. To solve this, Unity provides the ScriptableObject class: a specialized data container designed to exist independently of scene instances, operating fundamentally as a persistent project asset.

The Flyweight Pattern and Memory Layout Optimization

The architectural brilliance of the ScriptableObject class lies in its implementation of the Flyweight Design Pattern-a paradigm that focus its developing efforts in using common immutable data across objects of the implementation.

When configuration variables-such as base health or max velocity-are declared directly within a MonoBehaviour, those variables are fully duplicated in memory for every instance of agent in the scene implementing said logic. If a simulation requires countless game objects of the same

class, the memory allocation for those data fields scales linearly to the number of actors present of such class.

$$\text{Memory Total} = N \times (\text{Runtime State} + \text{Static Configuration Data})$$

By migrating static configuration data into a ScriptableObject, the data is decoupled from the scene instances. The ScriptableObject is compiled into a single immutable data block cached at the project asset level. When an agent is instantiated, it receives a reference back to that single shared memory allocation of such allocated memory. The memory equation is reduced to a flat constant configuration cost added to the variable runtime state:

$$\text{Memory Total} = (N \times \text{Runtime State}) + \text{Static Configuration Data}$$

This optimization extends to asset serialization.

Because ScriptableObjects are native assets, they are serialized directly by Unityâs YAML pipeline. They can be dynamically loaded via asset bundles or addressable asset systems, allowing the scenario to be updated, streamed, or patched at runtime without modifying scene files or recompiling source code.

Modular Architecture and Pluggable Behavior Trees

Beyond simple data storage, ScriptableObjects allow for the creation of pluggable, data-driven logic architectures. Because a ScriptableObject can store methods alongside data, it is possible to encapsulate modular pieces of logic which can be applied to the data contained. Methods that allow the static interaction of the variables, efficient search of parameters or simply getter functions can be implemented to augment logic features into the core data-containment purpose.

In a modern Behavior Tree individual nodes, actions, and decision-making criteria can be written as sub-classes of a base ScriptableObject. A game designer can then instantiate these classes to act as separate, abstract logic execution component. When an agent in the scene needs to make calculations based on the assets data and the current scenario it is in, it passes references of internal variables into the ScriptableObject asset's execution method. The asset processes the logic using its stored settings and returns a decision back to the agent. This design cleanly separates the data and structural blueprint from the physical actor executing it, allowing the developer to separate the way it is performed an action and the actual reason behind said action [43].

3.3 Structural Synthesis: The Separation of Logic and Assets

Architecting a production-ready system inside Unity requires a strict separation of concerns, synthesized by combining the specialization of both classes into a single ecosystem. The design must explicitly draw a line between the physical entity interacting with the simulation and the constant data structures that control such interaction.

Under this hybrid architectural model, the MonoBehaviour functions as the Actor. Due to its volatile, scene-dependent, and lightweight structure the role it takes is focused around dynamic, scenario-specific logic. It contains only the dynamic, real-time variables unique to that specific instance, such as its current health, its current position vector, its active destination path, and its target reference.

Conversely, the ScriptableObject functions as asset library and container and abstract logic executioner. As a consequence of its immutable nature at runtime, scene-agnostic, and shared across all instances of a specific agent type, it houses the broader behavioral logic, the algorithmic evaluation formulas, and the static code infrastructure that govern features of game object or scenario independent from the single instance.

```cs
using UnityEngine;
using System.Collections.Generic;
using UnityEditor.Animations;

[CreateAssetMenu(fileName = "NewEntity", menuName = "ScriptableObjects/Combat/Entity")]
public class Entity : ScriptableObject
{
    [SerializeField]
    private Stats baseStats;

    [SerializeField]
    private List<LevelUp> levelIncrease;

    [SerializeField]
    private List<SkillLevel> skillList;

    [SerializeField]
    private List<PhaseBehavior> phaseBehaviors;

    [SerializeField]
    private AnimatorController animatorController;

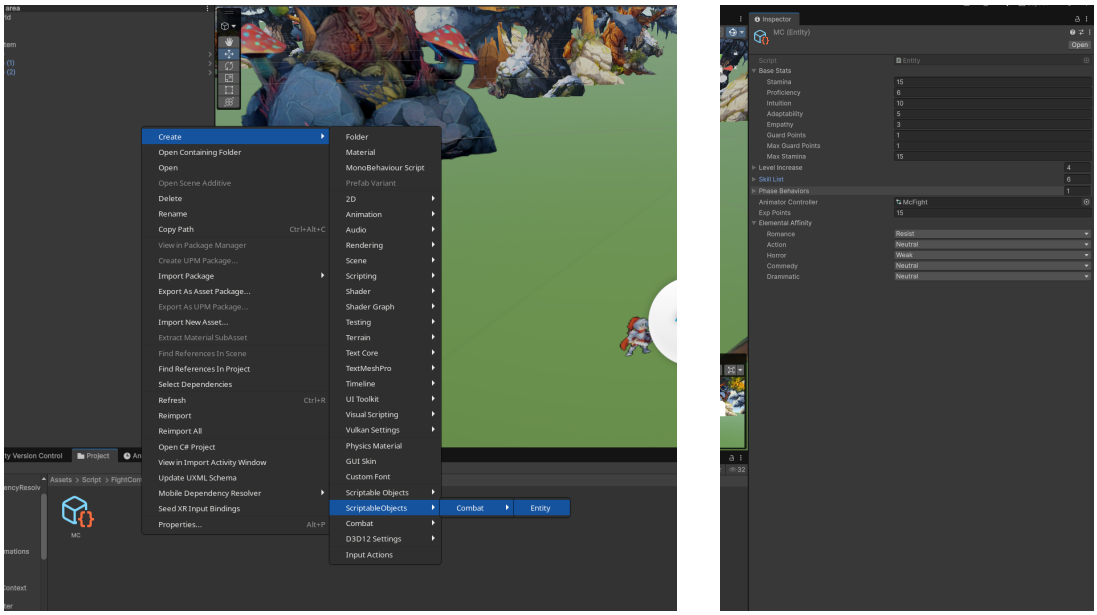
    [SerializeField]
    private int expPoints;

    [SerializeField]
    private ElementalAffinity elementalAffinity;

    public Stats BaseStats => baseStats;
    public List<LevelUp> LevelIncrease => levelIncrease;
    public List<SkillLevel> SkillList => skillList;
    public List<PhaseBehavior> PhaseBehaviors => phaseBehaviors;
    public AnimatorController AnimatorController => animatorController;
    public int ExpPoints => expPoints;
    public ElementalAffinity ElementalAffinity => elementalAffinity;

    public Stats GetCurrentStats(int level)
    public List<Skill> GetCurrentSkills(int level)
    public AnimatorController GetAnimator()
    public List<PhaseBehavior> GetBehavior()
    public Stats LevelUp(int baseLevel, int newLevel)
}
```

Figure 3.3: An example of script implementing the scriptable object class Entity



(a) Creation of an asset (b) View of the asset in the inspector view

Figure 3.4: Creating an asset starting from the Entity scriptable object class

Combining this two classes can create a complex system in which the MonoBehavior class governs the physic and visual implementation of the game loop, while the ScriptableObject class provides

If the Monobehavior class can identify for example the movement mechanics, the assets originated from ScriptableObject classes can describe the abstract elements of the mechanics like for example the movement speed of a specific kind of entity or its animation.

This combination allows to implement the game scenario logic by dividing the main behavioral patterns with factors completely independent from specific instances game object's behaviors like sprites, models and animations. Exploiting such organization to decouple functions and assets can increase the efficiency of the resulting structure.

3.4 Local Inference Infrastructure and Contextual Workspace Integration

To implement a prompting methodology without external cloud dependencies, this project exploits a local AI inference pipeline. Executing Large Language Models via remote APIs introduces severe architectural vectors of failure for real-time software development, including non-deterministic network latency, telemetry-based data exfiltration risks, and vulnerability to API deprecation or pricing shifts. By self-hosting the model locally, the generation, validation, and optimization loops are kept completely within the hardware boundaries of a single workstation, transforming the generative AI subsystem into a deterministic local compiler utility.

3.4.1 Ollama: the base of the self-hosted strategy

In the context of self-hosting strategies one platform is gaining traction in the recent years, thanks to its free and open-source nature.

Ollama is a platform that allows to download, run and manage open-weight LLMs locally on either personal computers or servers. Built primarily on C and Go-an high-level, statistically typed programming language designed by Google known for its simplicity and efficiency. Often refered as "Docker for LLMs", it allows to pull and run model with simple and easy commands-allowing to do both of those actions with one single command. Its functionalities are backed by the utilization under the hood of the llama.cpp library-an open-source LLM inference engine written in C and C++, born to run AI models without requiring Python or Pytorch. Thanks to this infrastracture developers and organizations are able to run LLM architectures without necessarily relying on external cloud interface-but keeping that possibility active for an higher scaling. The project focuses on data privacy and control maintaining models, prompt and responses under the user's security boundary [23].

The platform's story

Founded in 2021, the company behind it released the tool publicly in 2023 for the first time. The philosophy of simply accessible command-line tools was due to their prior experience in the creation of Kitematic-a tool designed to simplify Docker container management on MacOS. This initial launch was focused on releasing a version which could be ran on Linux and MacOS, but given the traction this platform got it brought to a Windows port of this infrastructure.

Some notable milestones include it's following implementation of hardware acceleration for both NVIDIA and AMD's GPU through their respective proprietary technology and the creation of an official GUI to chat.

Ollama's model configuration and quantization

Ollama's model utilizes the GPT-Generated Unified Format (GGUF), a specific format for storing and loading models. Chosen to replace the GGML format, it provides better compatibility, metadata handling and performance optimizations for quantized models.

The last characteristic is what makes possible to use massive models to run on machines with limited VRAM-Virtual RAM, the random access memory dedicated for GPU operations. By reducing the precision of the model weights to lower representation the memory usage and the computation time drammatically reduces. To quickly describe what the structure of the

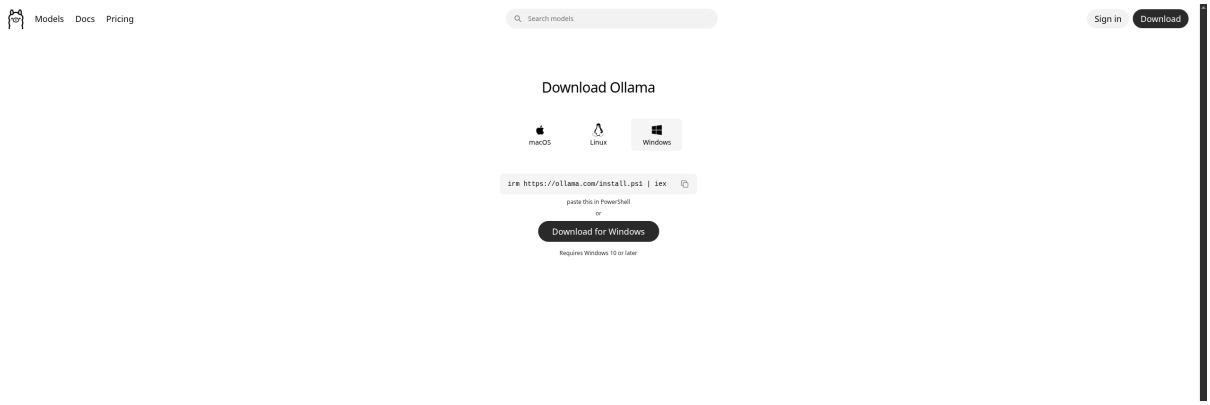


Figure 3.5: Ollama download page

model is a naming convention has been set up. While full precision, non quantized, models are named FP16-in floating point 16 bit representation-quantized models are named depending on the number of bits of the weight representation and the way said weights are scaled. The final structure of the name is QN_C where N are the bits for the weight representation and C is the compression method, being either K, K_M and 0. K stands for K-quants, an hierarchical compression system that groups weight rather than rounding them. The variant with the M stands for Medium precision, which acts as balance between quality and size by using more bits for the layers that impact the output the most. 0 stands for uniform scaling, in which one shared scaling factors rather than using groupings. The table 3.1 describes some of the quantization strategies that can be utilized.

Quantization	Bits per weight	Quality vs. Size trade-off
Q2_K	2	Noticeable quality loss
Q4_K_M / Q4_0	4	Minimal quality loss
Q5_K_M	5	Near full-precision quality
Q6_K	6	Close to full-precision quality
Q8_0	8	Large files, high memory use
FP16	16	Maximum size

Table 3.1: Comparative table between quantization strategies

Installation and setup process

The installation process is simple and quick, starting from the download page:

https://ollama.com/download

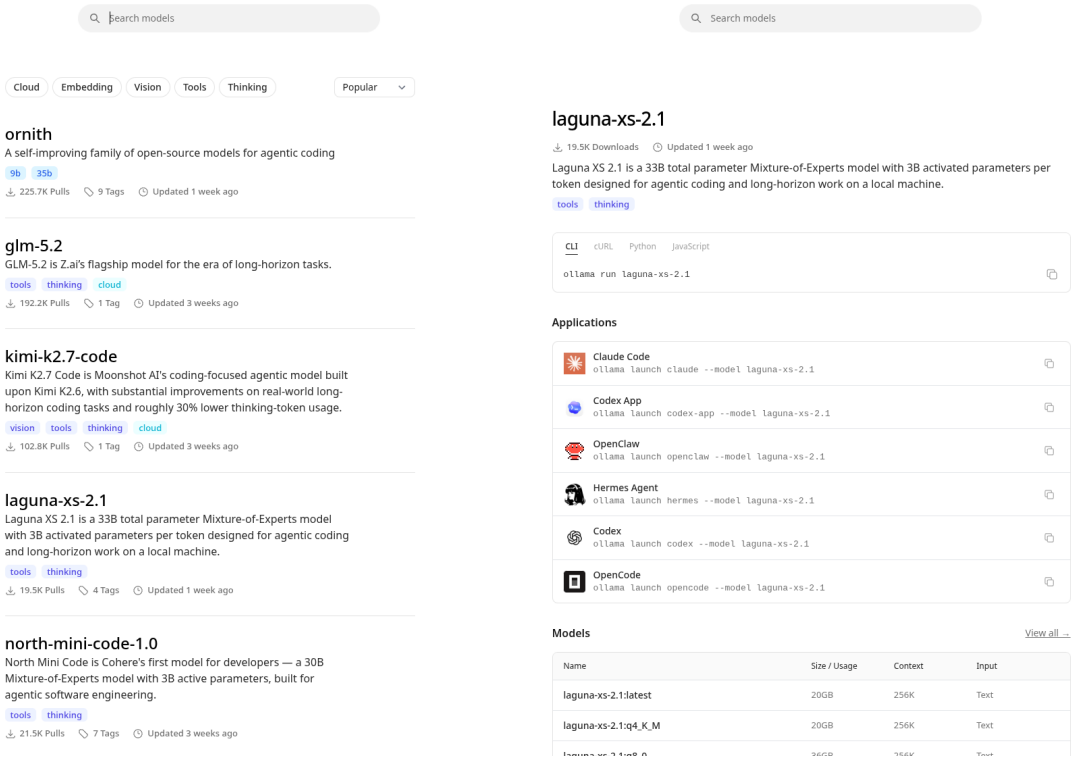
From this page either a command which can be pasted in the terminal or a GUI installer for MacOS and Windows can be used to install the platform on the device.

A quick start option after the installation can be used by launching the command "ollama run" followed by where the name of the desired model. This command will start the Ollama server and commence an interactive chat session with the selected model. Afterwards any time the machine is started to launch the Ollama server the user will utilize the command "ollama serve". By using any of the other commands available it will be launched automatically.

Before interacting with the inference model it must be downloaded. This step can be either be called with the command "ollama pull" followed by where the name of the desired model or by the use of the "ollama run" command-which will start by downloading the model and consequently start the interactive chat [1].

To find the models to launch the platform has the following website, which allows to find model based on the user's needs:

https://ollama.com/search



(a) Ollama's model research page (b) Example of ollama's model page

Figure 3.6: Example of downloading a model from Ollama's platform

This website lists all the model present available for the ollama technology and for each model multiple dimensions are available-making the choice of the model available for numerous types of hardware. In a model's page all the versions are available with a command ready to be easily copied and pasted.

While this technology has been created focusing on self-hosted strategies, the possibility of

utilization of cloud interfaces has been added. This not only improves the scalability of the infrastructure running the inference, but also lowers the burden on the terminal hardware. While being a strong improvement this implementation sacrifices one of the main pillars of the initial implementation, which is the user privacy, so a long and pondered evaluation must be done before deciding which strategy to use.

Not only self-hosted: additional use cases for improved scalability

While this technology was fundamentally engineered to maximize the efficacy of decentralized, self-hosted deployment models, the framework has evolved to support upstream cloud scaling. This hybrid capability allows engineers to offload heavy inference tasks to remote cloud instances, lowering the local compute burden while significantly increasing maximum query throughput. However, transitioning from a localized architecture to a cloud-hosted API necessarily breaks the user's isolated data perimeter, sacrificing baseline privacy guarantees and requiring a careful risk-reward evaluation before production deployment.

3.4.2 From Tongyi Qianwen to Qwen3-Coder: An Open, Code-Generation Focused Model

Developed by Alibaba Cloud, the Qwen family of LLMs is a lineage of models that evolved from foundational bilinigual LLM-model specifically trained in only two languages-into a multi-modal ecosystem utilizing the Mixture-Of-Expert (MoE) architectures-a specific type of network that breaks into specialized subnetworks called "experts", specialized in a set of tasks.

The origins of the model

The first publicly available model debuted in april 2023 by the name of "Tongyi Qianwen".

This release was further increased with the issue of the open weights for the 7B and 14B iterations, followed with the release of the 72B flagship closed model. Its core design was focused around the training on native bilingualism in Chinese and English.

The real jump in scale was initiated with the 1.5 update version. It's main achievement were the capability to shrink the memory footprint of the Key-Value cache and maximizing the inference speeds. This was made possible by employing a selective integration of the Grouped Query Attention (GQA)-where the query heads are divided into smaller groups, each sharing a single Key and Value head, effectively reducing the memory overhead while keeping the model quality [7].

The broadening scope

This modification was exploited and expanded throughout all scaling ranges of version 2.0, which focused toward broadening the accessibility enhancing multilingual capabilities. From this model on 27 additional languages were supported beyond the two original. Another important addition was the introduction of its sparse MoE configuration.

While expanding the flagship lineup to include a 32B model Alibaba deployed version 2.5 in september 2024. From this point the ecosystem branched from coding-specialists and reasoning models. In later iterations released in January 2025 focused on enhancements in the visual agents capabilities, allowing the model to interact with graphical user interfaces. The last improvement of this specific version includes its dense MoE in the form of the model Qwen2.5-Max, handling enterprise-scale workloads [22].

Version 3.0: the last step to until now

Its next evolution came with the Qwen 3 series, which restructured the foundation to be around hybrid sparse architecture and deep reasoning. This included the introduction a "thinking mode" into the inference loop, improving the navigation of complex prompts [31].

The utilized model

While the ecosystem has continued to iterate up to the contemporary Qwen 3.6 generation, this investigation focuses explicitly on a highly specialized code-generation variant within the baseline Qwen 3 architecture: Qwen3-Coder-30B-A3B, also designated in production environments as Qwen3-Coder-Flash.

This specific model focuses the entirety of its internal token weights and attention heads on the specialized requirements of software engineering, repository-scale comprehension, and agentic execution. By relying on an optimized MoE pipeline comprising 128 routing experts with 8 active per forward pass, rather than a massive, computationally prohibitive dense structure, the network matches the performance boundaries of much larger open-weights models. This parameter efficiency keeps its active memory footprint tight enough to facilitate localized, low-latency deployment on standard consumer-grade hardware configurations.

Unlike the general-purpose, reasoning-focused configurations of the standard Qwen 3 line, this coder variant explicitly strips away the intermediate "thinking mode" tokens to prioritize raw inference velocity and minimize time-to-first-token (TTFT) metrics. Instead of relying on natural language reasoning steps, the system's architecture was optimized using extensive execution-driven Reinforcement Learning (RL), training the weights directly against compiler feedback, test-suite validation matrices, and multi-turn terminal execution environments. This training methodology ensures that the model operates with an exceptional structural understanding of code completion syntax, making it highly effective at handling long-context function calling, tool utilization, and autonomous agentic coding within local IDE environments [2].

3.4.3 Interfacing with the model: from a chat-based to an advanced IDE-integrated experience

Once the Ollama server is running and the model's weight are downloaded, there are different ways to interact with the model in order to start using the LLM infrastracture.

As mentioned earlier, one possible way of utilizing the model is through the terminal and the ollama run command. Using this method created an interactive environment in which the user can write the prompt on the terminal and the answers are given directly in it. This is a simple and direct method but makes actions like tweaking the parameters, using system prompts or actually resuming past interaction incredibly more mechanic and slow. In some operating system like Windows this has been partially fixed by the ollama's official graphic user interface, which allows an experience similar to the chat-based models people are more accustomed to do and makes even easier to download models.

On other operating systems, like most Linux distributions, similar methods exist that allow to imitate the "chatgpt-like" experience. This is possible because the Ollama server session is accessible at a specific localhost (by default localhost:11434), allowing to receive requests and return the result of the computation of said requests.

One possible example of these interfaces is given by the open-source, multiplatform app which works as interface for the Ollama session.

It's capabilities include a wide range of main features, which improve drastically the user experience and the ease of use. The main capability is the possibility of customize the chat configurations-allowing to configure parameters like system prompt, model, and options (e.g., temperature, seed, context size, max tokens) for each conversation. These modifications can be

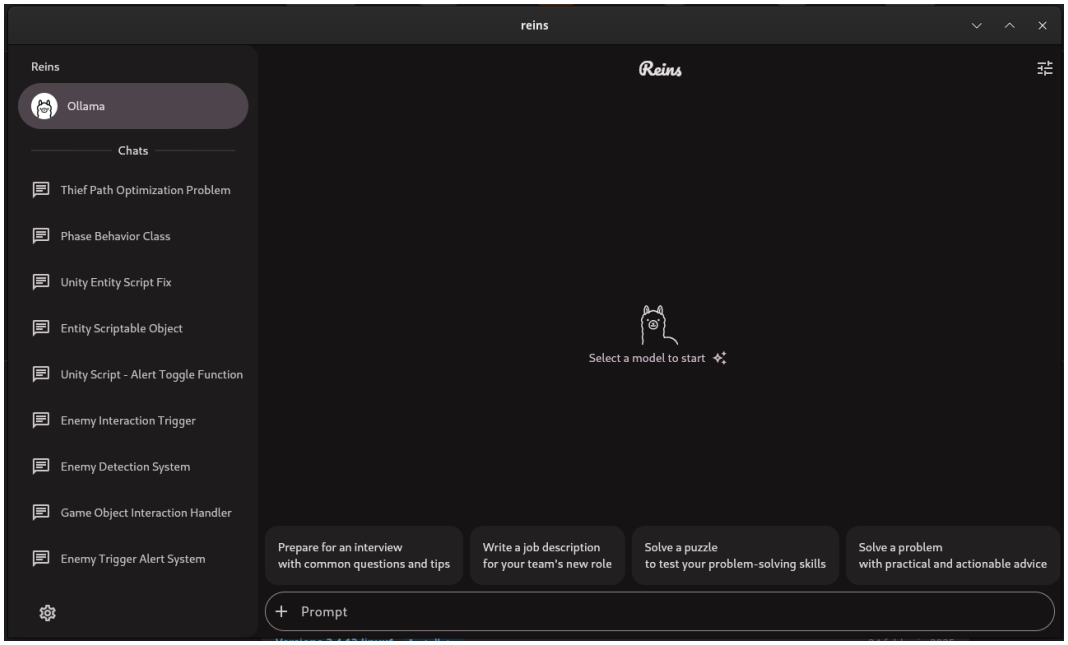


Figure 3.7: Reins interface

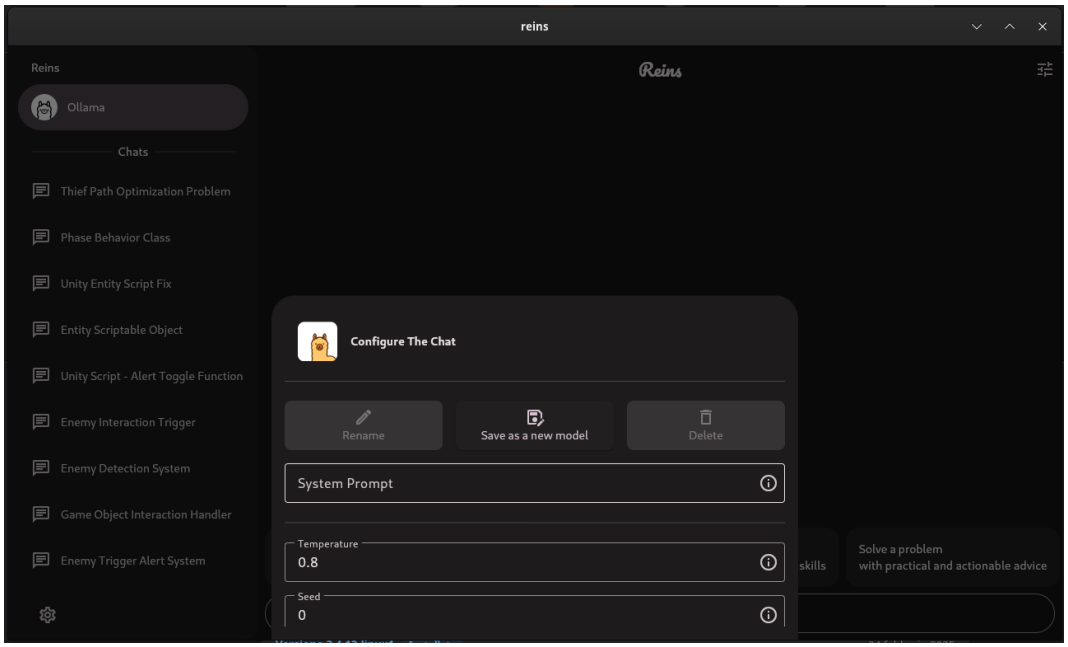


Figure 3.8: Reins model's parameter modification

done on the fly in any moment of the chat, allowing to adjust dynamically the settings of the conversation. Additionally if some specific settings become desirable as a model to standardize the flow this platform allows the capability to save such settings as new models.

Other features may include Quality of Life (QoL) feautures like the integration of images-allowing to both send and receive images-and to manage and switch between multiple conversations, all saved inside the device's disk.

It is available at the following link:

https://github.com/ibrahimcetin/reins

It is usable on apple devices through the App store or Linux distributions through Flathub-a store interface that allows the installation of program on Linux distribution using a Docker-like configuration called Flatpak.

Even given the great capabilities this interface offers one critical issue appears: the lack of a code generation-revolved approach. The chat-like nature of these structure structures creates points of friction caused by the necessity of swapping constantly windows.

To reduce these frictions moving the integration into the Integrated Development Environment (IDE) makes the utilization of these frameworks easier and snappier.

When considering an IDE and the integration of API that allow the communication with LLM models like Ollama multiple options can be evaluated. One possibility can be represented by recent types of platform that include or were built especially to exploit the use of AI in the process. Platforms like Cursor or Google's recent Antigravity shift the process from the software engineering to prompt engineering, being built on making code from the idea of mainly using both local and on-cloud agents.

While this approach can be good on paper this idea creates one great problem: it requires the programmer to switch both IDE and approach to programming.

Since one of the most used IDE is undoubtedly Visual Studio Code the desired approach is to use this platform. This is done so it is kept the initial familiarity of the environment, adding a smaller layer of complexity by integrating the novelty of the prompting approach.

In the context of the use of Visual Studio code, instruments with features similar to the ones previously presented are found in Copilot. Structured like a chat bot, this feature is integrated into the IDE to interact with the codebase and create or modify specific code sections. Notable additional functionalities included are the auto-completion feature-suggesting the additional code that can be used-or code editing feature-proposing changes that are automatically added in the codebase.

Unlike the previous approach, it allows to insert into the context snippets of the codebase that can be considered useful in the generation process. This additional characteristic creates the possibility of using the context to focus the result, reducing the risk of hallucination and redundancy.

While in recent implementations of Copilot it has been added the functionality to use local LLMs in the context of the chat experience, multiple tools gave the possibility of doing this before.

One example can be brought by the implementation of the extension "Continue" for Visual Studio code. This piece of software can mimic the activity done by Copilot, but completely shifting it to a local context. All the functions familiar in the context of Copilot are present and can be customized to the level of system prompts and model usage3.9.

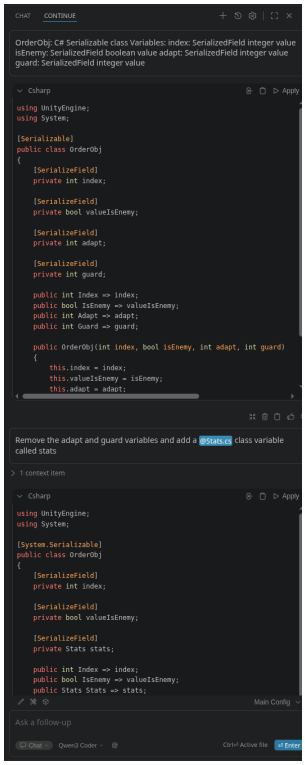


Figure 3.9: Continue's interface, located in the same section of the screen as Copilot

The only downside of the use of this extension is represented by its setup process. Unlike Reins' GUI setup or Copilot easier utilization, most of Continue's setup relies files called "configurations". These files list all the model that the context contains and the setup of the functionality each model can and cannot do, in addition to the system prompt. This setup is done for each model contained in the configuration. Using such setup allows to create standardized chat interfaces, skipping the initial establishment process of the LLM query infrastructure.

Approaching such environments for the first time- especially while finding specific settings- may become a difficult introduction to the extension. However once the initial friction is overcome it enables customization and standardization possibilities non-existent in the context of standard chat-based experience (Fig 3.11).



Figure 3.10: Continue's configuration creation, setting the roles and the system prompt for two models

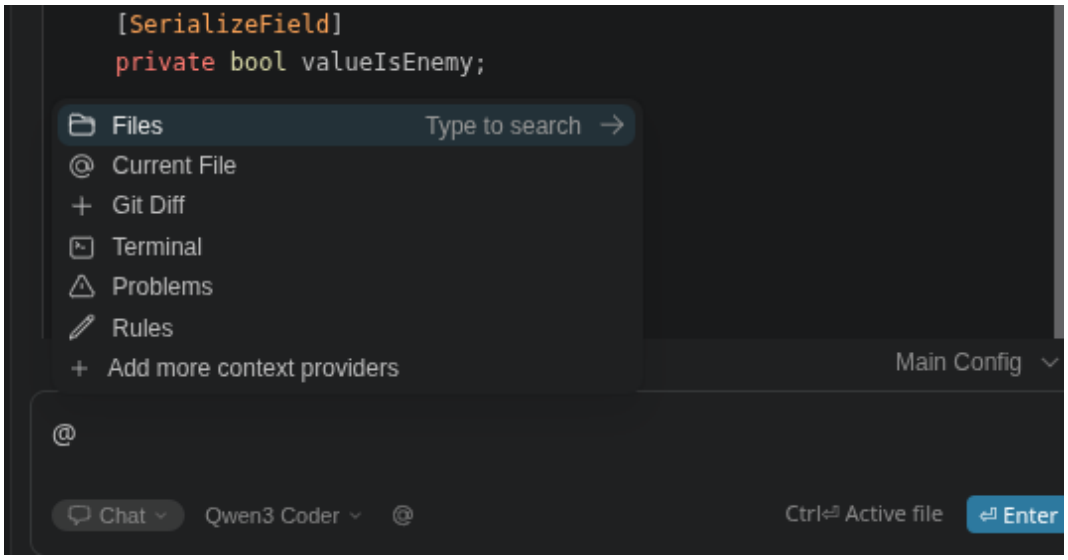


Figure 3.11: Continue's context selection menu, activating adding the "@" symbol in the chat environment

Chapter 4

Experimental/numerical evaluation

Given all the theoretical and technical concepts previously presented the framework methodology will be presented.

4.1 Methodology

The structure of the methodology of this study will be based in three main phases: a context and structure creation, the code generation phase and a final testing and correction phase.

This workflow is repeated for each game object component, as the target of this framework is generation of targets classes given a specific context in which it is inserted.

4.1.1 Environment and actors definition

This initial phase is the core of the framework, as it defines all the needed information that will be utilized in the next step and will be the foundation of the control stage. During the implementation of the framework this phase required to indicate two specific, macro categories.

The first main topic is formed by the ensemble of the general facts and notions that describe what the flow of the scenario, the "laws" of the specific game section that is desired to implement.

It is not enough to stop at the genre of the game, the setting or the characters. This steps focuses on breaking down each underlying mechanic into the most restrictive grouping is done.

Using a real example in the case of a game like "New Super Mario Bros." this process may divide the game in several main scenarios like the level selection map exploration, the level section that the player is playing and the menus that can be used throughout the game (4.1).



Figure 4.1: Game breakdown: example of decomposition from full experience to list of all possible game scenarios. Images from the Box Art [25], Level 1-2 [27], and Map of World 1 [26] of the game "New Super Mario Bros."

This first initial step is necessary to start to visualize and evaluate the general behaviors, specifically what the game is expected or not expected to do.

Once the scenario is defined the next step is to define what are the functional component of the environment are. The target of this step is, given one of the scenarios found in the previously stage, understand its underlying rules and exceptions must by breaking it down to its minimal elementary components.

This evaluations mustn't stop at the level of the game objects, because the game objects can have multiple game mechanics that revolve around them. Given the multitude of possible functionalities that each game object can utilize the last step is to group each similar group mechanic so that the final result is the list of the macro categories of the functionalities. For each macro category a specific class is generated.

Keeping the metaphor of "New Super Mario Bros.", focusing our attention in the level exploration section the scenario can be broken down in Mario, controlled by the player, the enemies, the destructible element of the game, the interactive elements of the game-like power-ups, coins and the flag-and the map itself. Focusing our attention even more on Mario's game object's mechanics it can be broken down to movement handling, animation and status handling-which indicates the current form and any transition given by a form change.

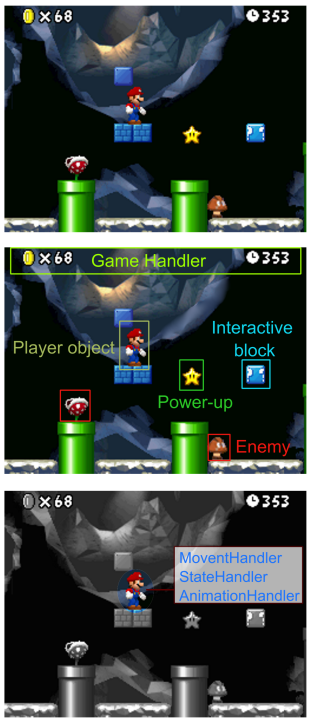


Figure 4.2: Breakdown of the implementation details: the methodology of this study applied to an example of game scenario. Image from level 1-2 of the game "New Super Mario Bros.". [27]

4.1.2 Multi-layer prompt generation phase

Multiple approaches have been evaluated to tackle the specific task of prompt engineering. As previously mentioned, the main challenges will be the concept of hallucination and redundancy. These difficulties added to the necessity of creating deployable-ready code capable of executing without creating actual problems makes the creation of the prompt the fundamental phase of the process.

An initial, tentative approach that was tried during the early stages of this study was creating an exhaustive explanation of the class implemented. This required quite specific prompt description which can become pretty difficult to manage-especially when the dependencies of the resulting class involved custom classes-, causing hallucination and making the process in itself inconvenient compared to the standard manual coding approach.

In order to mitigate the difficulties of the setup process and reducing the risks of hallucination and redundancy, a multi layer framework has been formulated. The objective is to gradually narrow down the context of the generated class, to the point of defining its role and the requests of the class, like shown in Fig.4.2.

Level 1: Scenario definition

Before evaluating the functionalities the single class needs to implement the context must be evaluated. All the element that need to be present to describe the specific game scenario must be stated.

The target of this section is not specify a particular functionality, but to describe in detail the context in which the requested component is put in.

By inserting the guardrails that limit the generation context the probability of hallucination is reduced, cutting down the possibilities of having unwanted features implemented.

Using a similar approach it is also diminished the likelihood of having redundant code, evading to have the creation of the implementation of external class or methods that can are implemented by other classes.

By giving the full picture of the game scenario, the model is restricted from implementing mechanics that are not necessary to the scenario or that implement unwanted rules.

This introductory part is present with the role and task definition of the LLM in the system prompt, as it is independent from all the elements of the context.

Level 2: The game object structure

While being the shortest piece of the whole prompt structure, it reflects an important aspect of the implementation.

Even if the code is syntactically correct and it reflects mechanics considered in the whole game scenario, it can't be ignored the fact that the class that implements a specific mechanic are linked to specific game objects.

This means that the code not only requires to be conform with the specific logic the developer envisioned, but must be also compliant with the game object configuration.

To put the importance of this section in perspective let's take for example the Collision component, responsible for object interaction. This component has a shape can be set as a physical object-which acts as solid object that create an interaction when its boundaries are touched-or as a "Trigger"-a traversable area which creates an interaction when another object enters in it. These two configurations do logically the same action-register a collision of an object with another-but if the model doesn't know which configuration has been set it might generate code that won't work, being logically incorrect.

Level 3: The class specifications

This specific section is what describe specifically the actions that needs to be implemented.

To guide the generation in the desired direction, this section must describe all the elements needed and how each of those parts work in the context of the required functionalities. The main description will include the variables that are needed and the methods that must be implemented. Given the strict structure that the rest of the prompt possesses the final description can be more loose and focused on the functionalities. For this reason the only required explanation is the general flow of the function, its input and output variables.

This represented a simplification factor especially comparing the initial approach tried. Being limited to the description of the single class that needed to be implemented, all the function needed precise indication to avoid hallucinations.

Exploiting Continue's capabilities: context enrichment process

During the testing phase and the creation of the demo both the Reins and Continue's environment were tested. This was to test the difference in the utilization and how much beneficial an implementation like this be.

The core advantage of the utilization of this extension rather than just a chat interface is the capability of gaining important information about the codebase. This can be done inserting the class in the context of the LLM by selecting the required files that are needed to implement correctly the request. To do so inside the chat the desired class or folder can be selected inserting the symbol "@", which will open a selection menu to help to choose the desired element.

This specific function allows the model to access previous implementations, completely avoiding the not so trivial task of explaining the role of each external class.

Complete mechanic generation approach

While creating the class two specific approaches were tried, taking inspiration from the two different types of software engineering process previously mentioned.

The first methodology took inspiration from the waterfall process. Its development aimed to envelope all the core functionalities of the requested class. Instead of dynamically changing the class while growing the project the scope of this methodology was to create the class requested capable of doing all the tasked functions ideally in one go. To do so one structured, detailed prompt which explained in an exhaustive way all the demanded features.

While this methodology had the advantage of having component ready to be tested directly in the game engine environment, it became increasingly difficult to keep track of all the possible functions needed in more complex implementations-especially considering the window of context of the model.

To tackle this challenge an alternative method was used which was born from the influence of the waterfall method. In this methodology the whole structure was used to create a base structure to implement a part of the functionalities requested. Successive prompts were utilized to add consequent implementation details to the resulting class.

The main benefit of this approach is seen in the slimmer prompts, especially considering the presence of context reference to reduce the explanation needed. However, while the use of multiple prompts generally determine a lower grade of granularity in the demands given to the model, this could increase the probabilities of hallucination in the long run.

Ultimately a balance must be found between these two methodology.

4.1.3 Logic validation and correction

When evaluating the quality of a code implementation stopping at the level of single file can hinder the understanding of the quality of an implemented logic. While fundamental to catch syntactic and structural errors of an implementation the core of the evaluation must be shifted to the whole scenario, especially when considering the collaborative nature of the components inside a video game.

To approach such evaluation tests are performed directly in the game engine's environment to study the interaction within the classes. To allow the testing phase to be performed as soon as possible dummy testing classes were built ad-hoc to support the execution of the generated classes.

After the creation of the testing environment is concluded the general tests are performed. Such tests usually involve directly simulate the scenario that is wanted to experiment. Given the results of the test, if any anomalies are found, the developer analyzes the logs and the code implementation in order to pinpoint the origin of the error. Once the nature of the problem has been identified the solution is formalized and sent as request to the LLM to audit the code implementation to fix the problem, effectively becoming a Human-In-The-Loop (HITL) process.

Rather than having the previous rigorous and structured prompt these modifications are explained through descriptive explanations that express the direct change in the specific code. The success of these direct, non-verbose way of express the request is made possible by the fact that the requests are mostly targeted at short sections of single files, allowing to be specific without prolific explanations.

4.1.4 Case study specific setting

Before defining the singular contexts of each scenario of the target implementation of this study, it is worthwhile mentioning the role and the specific rules the LLM was put to work with. As mentioned earlier the scope of the model's generation is being a direct translator of the design implemented, describing the underlying structure of the input.

In order to debug the code better the correctness of the flow the model should generate as much log messages as possible, to make possible to pinpoint in an easier manner the origin of any potential problem encountered. However it is also required to try making sure that no if statement with multiple conditions gets broken down in singular conditions just to log every single parameter.

The final system prompt used in this case study is the following:

"You are an expert in code generation classes used in Unity engine projects. You are given a context which will contain the whole scope of the game scenario to be implemented which will be the high level boundaries of the scope of the implementation. ADHERE with this context. Additionally you are given either a design for a specific component that is internal in the context and the game object that will contain said component or a modification to be made. If the design is given write the code implementation for such design. Focus on DECOUPLING. Add debugging log lines inside the implementation, but AVOID fragmenting if conditions to do so. If a modification is requested respond ONLY with the specific parts of the code to be modified and WHERE to put them."

This structure becomes the base to build the project, which will have a specific structure.

The designed game is a small project that blends turn-based combat with exploration mechanics, which represent the two main scenarios studied in the case study.

The target of this study is to create a simple environment, trying to be a good ground to evaluate the quality of the method utilized.

A great importance is given to the creation of decoupled components reusable in contexts that aren't limited to the envisioned specific use case. In order to achieve such objective the focus is shifted on the creation of a portion of the game experience, rather then creating a complete project.

Given the specific structure of the game this framework will implement, two main scenarios can be created separating the main components of the whole projects: an exploration and a fight context.

The first scenario, the exploration context, is focused in moving in a 3D scenario to reach other zones and enter combat with enemies.

The player controls a character that can either walk or run or interact with certain objects present in the area. Once the player is able to initiate the interaction with such elements a GUI window will indicate it. The interaction can range between messages, event to commence combat or being moved to other areas.

While exploring the area it can have one of three states: normal-the default state-, hidden or discovered. The state can change depending on what happens during the exploration.

In some areas enemies will be present, looking around for the player. While the player is not found they will patrol the area around. Getting near enough to the enemy will alert them. While alerted these entities will start looking around. The player gets spotted if stays within their field of vision for a set amount of time.

If an enemy spots the player its state will be shifted into discovered. In addition of having the enemy chasing the player, being spotted by other enemies will be easier as it takes less time.

Some areas allow to hide the player-putting it in the hidden state-, making it less visible. This makes exploration easier, since enemies will take longer to spot the player.

If an enemy touches the player after discovering it combat initiates with the enemy being in an advantageous position.

On the other hand if the player gets near enough a "sneak attack" interaction will be available. Initiating such interaction will start combat with the player having the advantage.

After a certain amount of time is passed the state will be reset to normal state.

The description of the exploration context used in the first level of the structured prompt is

the following:

"The scenario will be an exploration scenario. The player uses a character which can be either walking or running. The character can be in one of the three state: normal, hidden or discovered. To change from the normal state each state has its condition. The hidden state requires to be in normal state and to interact with specific objects. The state has a duration and after it ends the state returns to normal. The discovered state occurs when the player either enters sight of an enemy while being in normal state or staying in the sight area for a duration of time while being in hidden status. This state returns to normal state if no enemies can see the player for a certain amount of time. If the player walks or runs around enemies but not directly seen by them while in normal state the enemies will have a wider area of sight and will start looking around more quickly. Interaction with object triggers events. Events can have different natures: dialogues, achieving requirements, change scene and/or initiating combat. Combat is NOT part of this context. An event can have one or more of the previous actions. To interact with an object or an enemy the player must be close enough to it, a ui element will notify the player if the interaction is possible. If the player in normal state interacts with an enemy combat will be initiated with the player advantage. If direct contact is made with the enemy and the player is in discovered state the combat is initiated with an advantage to the enemy team. UI elements consist of multiple types of elements. A bubble speech is used to say things to the player. Some blocks set are to notify the player that interaction is possible. By pressing the pause button the pause menu will open. The pause menu is NOT part of this context."

The second context-the fight context-focuses in the turn-based nature of the game. This context starts whenever the player initiates combat-in both the case he/she has been ambushed or has attacked the enemy during the exploration. At the start of the fight, the side that did the ambush gets an extra turn.

Each game entity possesses 5 main statistics, which have an alternative naming to the standard RPG statistics: stamina-which represents the hp statistic-, proficiency-the equivalent of the attack statistic-, intuition-which acts as the mp statistic-, adaptability-that has a behavior similar to the speed statistic-and empathy-which is a surrogate of the defense statistic. The use of this peculiar naming convention was a consequence of the initial idea of the complete game in which both the exploration and fight contexts are put, that considered social events and real world activities.

The active elements in the scenario-being either the enemies or the player-act during their turn by either attacking normally or using a skill. In order to reduce the difference in the implementation between the two normal attacks are considered a special kind of skill that any entity possess.

All skills have two types of multipliers, that are used in different situations. The first is a damage scaling multiplier-which indicates how much each statistic is involved in the damage output-, while the second indicates the debuff-temporary reduction of the statistics of the target-that this skill inflicts. These multipliers are structures that contain one value for each statistic.

In order to make the project more similar to other turn-based experiences, two other mechanics are added-an elemental affinity system and the concept of "guard points". While the first concept is more straightforward-giving a nature to each skill which can have an higher or lower effect depending on the target hit-the second is an extension of this concept. Each entity has a set amount of those points, which are diminished by one every time an attack which has a weak affinity is suffered. Once the points reach zero the entity cannot act until the points are not filled again. Every turn in the "stunned" state-this situation of impossibility of action-2 points are regenerated. Additionally some skills can return some points.

During combat an enemy can have more than one phase-period of game play in which an

enemy act in a specific way. Shifting between two phases can mean not only change in how the enemy behaves, but also what kind of moves are used. The change between one phase to another occurs when reaching a stamina treshold, generally expressed in percentage.

To end the context, either one of the two part has to be felled. If the player wins the game returns in the exploration context, otherwise the game ends.

The description of the fight context used in the first level of the structured prompt is the following:

"The scenario will be a turn based fight scenario. Entering this scenario there is a static class which contains the values needed to enter the scenario: the list of enemies and the advantage and the four members of the player's team. The advantage will be used to decide if one of the sides gets an initial extra turn. The advantage can be for the player (the team gets an extra turn), for the enemy (enemy gets one extra turn) or none. Each entity has the following stats:

- Stamina -> HP (Health Points)
- Proficiency -> ATK (Attack Power)
- Intuition -> MP (Magic/Skill Points)
- Adaptability -> SPD (Speed / Turn Order)
- Empathy -> DEF (Defense Rating)

Once the first extra turn part is checked the order is decided through the adaptability stat. Each turn the entity can choose to either attack normally or use a skill. Each entity has skills which can have a cost in term of Intuition. Each skill can be physical or magical. Each party member can assign a maximum of 8 skills between the one unlocked. Each skill has a type. Skills are unlocked at specific levels. Each entity has his own set of weaknesses or resistences. Depending on the type interaction damage can be increased or diminished. Skill can buff or debuff entities. Debuffs are not effective if the target is resistant to said skill's type. Each entity has a set number of 'Guard points'. When receiving an attack whose type is a weakness a guard point is depleated. If guard points go to zero the entity will be in stunned state. In stunned state the entity won't be able to do anything and will regain 2 guard points. When all the guard points are recovered the stunned state is cleansed. There are skills to recover guard points. Every time an enemy looses all Guard points the player receive an Encore points. Encore points are used to use Encore skills. These are special skills that are exclusive to character and have special visual effects and effects in combat. Enemies can either have one or more phases. Each phase has its own moveset, in which each skill have a probability of being used. The battle ends when the enemy side is completely defeated or one specific character of the player team is defeated. In the first case the exp gained is distributed to the members. If a team member levels up it is shown, along with any possible skill unlocked. Then the player is returned to the point he was in the exploration context. This part is NOT included in the context. In case the team member is defeated a loosing screen is shown."

In both cases a meta-prompting strategy was used by giving the full description of the scenario to a LLM. As a result the previous descriptions are obtained and manually refined.

4.2 Numerical results

The scope of this section is to express how the specific study case evaluated was evaluated, specifying what are the interesting metrics that are used. Consequently the empirical results are shown and explained, adding any ending considerations.

4.2.1 Evaluation of the empirical results

To evaluate the results of this experimentation the metric used is the prompt distribution rather than the actual codebase result.

The underlying factors of the evaluation

The reason behind this decision is brought by three main factors that allowed a result that, given a correct logic, would produce a result capable of running the correct logic "out of the box".

Two of the main factors originate from the model employed to work in such task. The nature of the model used-given its 30B parameters-provided a deeper network which was capable of producing superior representations.

Furthermore the training methodology is another characteristic that impacted in the model's output quality. Relying on a model focused in domain-specific tasks rather than general-purpose work provided ending results of higher quality as a consequence of its familiarity with the code syntax, structures and language-specific design patterns.

Additionally the utilization of a rather strict and guided prompting strategy eased the work of the model during the inference of the desired code implementation. The methodology employed represents an approach very distant to a strategy akin to vibe-coding applications-characterized by having the prompter as a passive operator describing the outline of the desired implementation, generally leaving the model to fill in the blank in terms of technical implementation.

In this approach the developer indicates the structures, functions that must be present in the generated final result. This characteristic, combined with the specificity of the prompt structure and the additional possibility of context-augmentation through the use of the Continue extension, impacted positively in the outcomes of the proposed framework.

Evaluation framework

As a consequence of the previous considerations the result where evaluated by employing three parameters, which describe the relationship between the prompt and the context of formulation:

- the number of "constructive" prompts. This parameter indicates the number of prompt utilized to implement new code, both in the case of implementing new classes and in the scenario of implementing new source code to increase the potentialities of a class. This variable is the only indicator of a "positive" outcome as it grows compared to the others, because it indicates only that the developer based its implementation process on incremental methodologies rather than implementing all the required functionalities with one request. It is worth noting however that by increasing such quantities, especially when managing complex relationships, that the probability of hallucination increases.

- the number of "self-adjustment" prompts. This attribute describes specifically the case in which the model generates a code implementation compliant with the developer's specifications and yet resulting in a erroneous implementation of the desired logic. This parameter indicate that developer is paying the technical debt that was originated from a flawed initial design. The lack of deep understanding in the implementation details, dependencies or simply errors in the course of the underlying logic are the main culprits of these problems. While indicating that work has been done in the wrong direction it doesn't directly undermines the value of the framework, as it is not the direct cause of the situation.

- the number of "de-hallucinating" prompts. This category of argument is mainly focused on the concept of adjusting all the problems formed by hallucinations and errors in the code generated by the model. Although an increase of this parameter implies that the model has an higher likelihood of hallucination, its value doesn't necessarily correspond to the

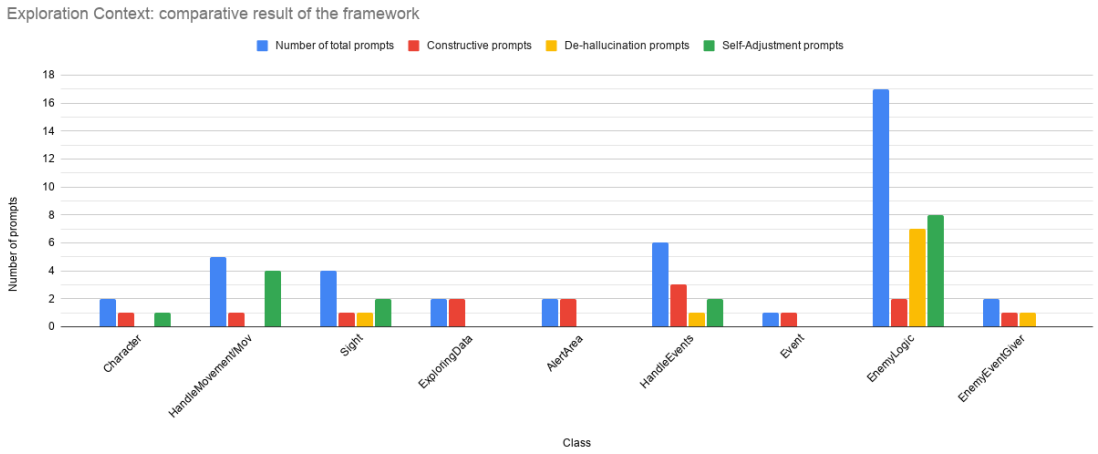


Figure 4.3: Exploration Scenario comparative data

number of hallucinations that occurred during the generation. The increasing value of this argument simply states the intensification of the efforts needed to spot and correct the results of incorrect inference.

4.2.2 Empirical results

The project repository, containing the code of the implementation and some examples of chat sessions, is present at the following page:

https://github.com/Francesco-Re/BlockFighter

Out of all the code implementations present inside the repository 24 classes were the result of generation. Throughout the 149 prompts needed the highest amount of prompt focused in the refinement and adjustment phase. The vast majority of the rectification process revolved around the "Self-adjustment" process.

This indicates that the greatest factor that influenced the effort in the prompting labor is the developer's lack of consistency in the global structural mechanics, either caused by errors in the design's logic or change in the implementation details. Despite the fact that this doesn't necessarily depict the superiority of the methodology proposed, it indicates that the extent of the work isn't focused on adjusting the errors caused by the framework's incapability of generating high quality, adherent code. Such conclusion allows to indicate that the framework allows flexibility to the developer's method, which is able to use the structure to create an expansible and working code base. As the context is restricted by the nature of the scenario and the technical implementation of the single object-which are mainly a consequence of the creator's choice-the design of the how the desired mechanic is implemented is completely up to the developer.

A complete and detailed description, containing all the values is present in the Tab. 4.1. The elements presented in the table are the classes created in order of implementation. This detail is important to understand the distribution of the values, since all the elements prior the EnemyLogic class implementation were created using the waterfall-like approach while from that class afterwards it used the agile like approach. In order to better visualize the parameter distribution per each class the values are put into graphs, divided between the two context of origin (4.3 indicates the values of the classes from the exploration context, 4.4 for the classes of the Fight scenario).

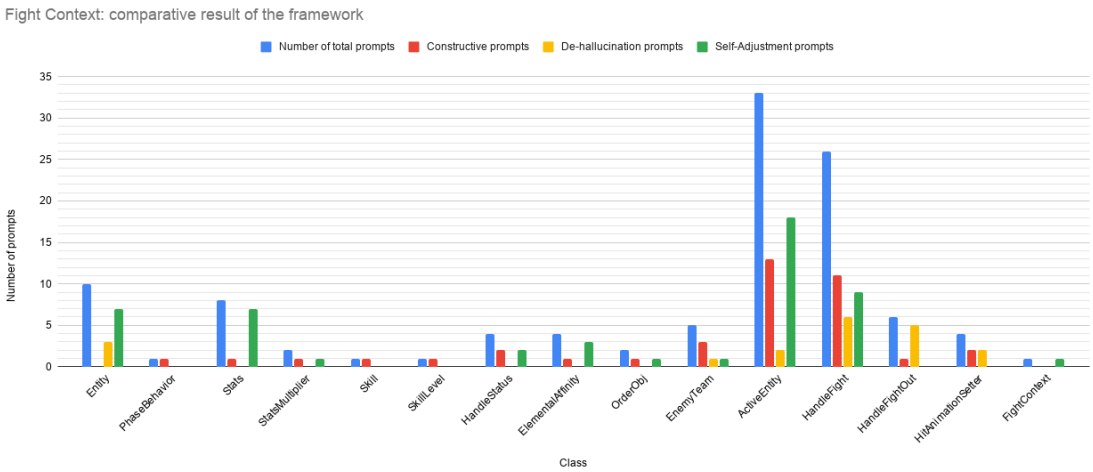


Figure 4.4: Fight Scenario comparative data

Table 4.1: Table of experimental results

Class	**Number of prompts**	**Prompts causing hallucinations**	**Constructive prompts**	**De-hallucination prompts**	**Self-Adjustment prompts**
Character	2	0	1	0	1
HandleMovement	5	0	1	0	4
Sight	4	0	1	1	2
ExploringData	2	0	2	0	0
AlertArea	2	0	2	0	0
HandleEvents	6	1	3	1	2
Event	1	0	1	0	0
EnemyLogic	17	0	2	7	8
EnemyEventGiver	2	0	1	1	0
Entity	10	1	0	3	7
PhaseBehavior	1	0	1	0	0
Stats	8	0	1	0	7
StatsMultiplier	2	1	1	0	1
Skill	1	0	1	0	0
SkillLevel	1	0	1	0	0
HandleStatus	4	0	2	0	2
ElementalAffinity	4	0	1	0	3
OrderObj	2	0	1	0	1
EnemyTeam	5	1	3	1	1
ActiveEntity	33	4	13	2	18
HandleFight	26	2	11	6	9
HandleFightOut	6	0	1	5	0
HitAnimationSetter	4	1	2	2	0
FightContext	1	0	0	0	1
Total	**149**	**11**	**53**	**29**	**67**

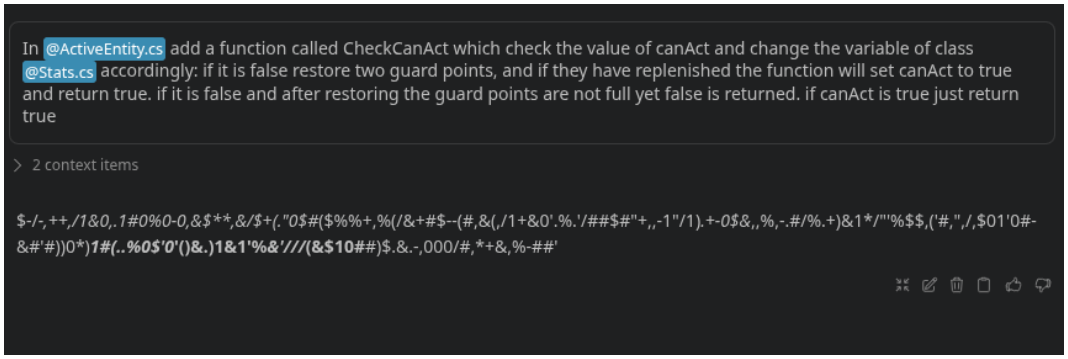


Figure 4.5: Limitations of the self-hosted approach: corruption of the model

4.2.3 Main criticalities

Despite the result of the experimentation indicates the capability of the framework's validity in a programming environment, some critical aspects must be evaluated. While some are determined by the implementation details of the structure such as the model or the hardware used, a key factor is determined by the developer's capabilities.

Hallucination: a never ending issue

One first detail is determined by the model's accuracy in the implementation. Despite the relatively big scale and domain-specific LLM, the concept of hallucination wasn't completely eliminated. Granted that it was mainly represented in a limited fraction of the result prompts, its presence created friction not only to introduce new features but also to find the logical fallacy of the current implementation.

This aspect generally grew in frequency the more the single chat increased dimensions. The main strategies used to tackle this challenge were the use of multiple chats and the separation of each implementation in multiple requests. Although the consequence of the strategies was an improved accuracy it came to the cost of an increased number of prompts to explain certain concepts multiple times and an higher design time. Additionally it's worth noting that it is not always a feasible strategy, especially in larger implementation that required bigger prompts to encapsulate all the complexities of the strategy.

Hardware limitations

In the context of the study the machine utilized is another potential point of failure. Despite the fact that the hardware used allowed medium to high level of performance-thanks to its 32 GB of RAM and 16 GB of VRAM-the utilization of this machine as the only base to work on the whole infrastructure determined some limitations. The presence of multiple programs open during the work environment-Visual Studio code, the ollama Server and Visual studio code along with all the possible supporting programs-occasionally directly impacted the results. This limitation has been represented mainly by the complete corruption of the inference of the model, resulting in complete randomic outputs as shown in Fig. 4.5.

While some mitigating factors involved reducing the amount of context provided in the prompt and occasional reboot of the Ollama server, the straightforward solution is changing the model to a lighter alternative.

However using a similar solution can cause consequent deterioration of the framework's result, for this reason the trade off between performance and cost must be carefully evaluated.

The innate limitation of the framework

The last type of limitation is completely independent from both the model and the hardware, but it is born from the approach's nature.

The original idea behind this methodology was to integrate the capabilities of the LLM in the process as a tool for improving the development process, rather then using it as a surrogate of the creator's work. The main target was to support the development process by making up for the personal lack of knowledge or simply for avoiding to directly write the codebase. To do so the model was forced to be a mere translator, rather than an advisor or an assistant in the process. This gives an enormous importance in the abilities of decoupling each rule and decode it appropriately. The consequence of these considerations is the fact that the framework doesn't include any kind of consideration to the errors in the engineering process, rather then just additional technical debt. Effectively this means that if the design is flawed in any way the model is instructed to implement it anyway, without actually checking the validity of the logic. The effect is an erroneous implementation that can be verified only in testing, causing not only technical debt but also to waste additional time to pinpoint the cause of the logic error.

This factor is amplified especially considering elements like libraries, component and syntax specific to a certain game engine, requiring even more careful thought process.

Chapter 5

Conclusion

The scope of the study is to provide a framework capable of being used as integration of the coding step in the context of game development. Rather than using LLMs to replace the engineering and development process altogether, the model used is mainly inserted as a design translator. Instead of receiving the design for the whole game that the developer wants to create, the model's work is focused in implementing the single components of the experience. the final product is broken down following the normal process of game development, once the basic component are decided the design is written and then used to generate the final result.

The process of creation starts by breaking down the game into scenarios-environment with a set amount of rules, usually indicating specific game mechanic or game loops. Once the scenarios are defined the singular component element must be identified, defining each game object and how which mechanics need to be implemented. In the context of the game engine used-which is Unity Engine-each game object can have one or more scripts, called components. For each component one mechanic is implemented. Once the classes needed are defined the generation of the class is performed.

The prompting strategy used in the context of the class codification is a multi-layered approach. This methodology is based on describing each component with a gradual increase of specificity, in order to restrict the context of the application while defining all the dependencies needed.

In addition to the system prompt 3 levels of information are given, which gradually focus the scope of the implementation:

- Level 1, the scenario definition: defines the complete boundaries of the scenario, giving clear instructions on what should and shouldn't happen in the context.
- Level 2, the game object definition: describes the characteristics of the game object that contains the component class
- Level 3, design of the implementation: describes the actual C# class that needs to be implemented

Once the generation is over the code result of the generation is tested and refined if necessary, characterizing a HITL approach. During this phase the code generation can be use to either fix any code anomalies or integrate additional functionality.

The model used to generate the code base is Qwen3-Coder 30B, used through an Ollama server and interfaced through either a chat environment like Reins or through the Continue extension for Visual studio code.

Using a very specific framework, along with a powerful, domain-specific model resulted in having for most of the resulted prompt code that was executing without any compilation error. For this reason rather than analyzing the code quality the distribution of the prompt is the decided metric of evaluation of the framework.

Each type of prompt is classified as one of the following three categories:

- Constructive prompt: prompt that was used to either implement new class or integrate new features
- Self-Adjustment prompt: prompt that was used to patch any logic error caused by design flaws, completely independent from the model or the framework implementation
- De-Hallucination prompt: prompt that was used to patch any error due to hallucination

The result of the evaluation showed that the majority of the prompt were focused on Self-Adjustment prompts, followed by Constructive prompts and finally by De-Hallucination prompts. This distribution shows that most of the developer effort was focused in refining and patching the actual design, rather than fixing the code implementation result of hallucinations.

While reducing the hallucination this element was only mitigated but still present. This aspect, along with the testing in-engine, made the discovery of errors increasingly complicated. The framework additionally showed its limitation in two other aspects, mainly due to it's formulation nature. One limitation was due to the prompting strategy, which didn't allow the model to check the code implementation for error in the logic of the designed codebase. Another limitation was due to hardware specification. Despite the testing platform used provided medium to high level of computational performance, the computational load of the model was high to the point of occasionally corrupting the output result.

Overall the proposed framework results gave inspiring insights on how the LLM technology can be applied and integrated in the context of a software engineering environment. Despite the interesting results the framework has some limitations, mainly present in form of hallucinations, the non-existent error control caused by strict role given to the model and hardware limitations on consumer-level architectures.

Given the results of the evaluation, a few consideration can be made on the potential future of a similar approach. The strengths of this model proved by both the output code quality and the prompt distributions can be exploited by expanding the starting framework, using the current limitation factors as catalyst for improving the current methodology.

To tackle the hallucination factor and the restrictive nature of the model's role in the framework, an agentic architecture can be a viable starting point. The possible implementation can be built around the framework, allowing to break down the process similarly to how it was handled in the GameGPT paper. Exploiting structures in similar manner can help divide the load of the whole procedure into different phases. This subdivision can be used to leverage on prompting strategies specifically for each phase. For example techniques like RAG-that can be used for example by introducing the engine's documentation-are viable strategies to further increase the quality of the inference results.

A possible structure may be composed of multiple models starting with an observer model, which analyzes and evaluates the core logic of the design so that possible early errors can be found and modified collaborating with the user. It is worth noting that such approach would need to have the model knowing what is the mechanic that the developer want to be implemented along with its implementation design, adding significant friction to the methodology. To increase the efficiency a model in between can be added to implement a meta-prompting strategy. Converting the design obtained from the previous exchange to be expressed into a simpler, more concentrated manner the output can be made tailored for the coder model that has to implement the design. Using a model capable of extracting the required features from the context and the design to create a shorter, token-efficient representation of the developer's implementation plan can be created, offloading some of the interpretation process to the coder model. This structure implements the same core logic of the original framework-the added steps in the agentic architecture are done by the programmer in the proposed framework-, but adding some assistance capabilities to reduce the proneness to errors while designing the logic.

While these element proposed can directly mitigate two of the three main limitations previously mentioned, it might also help lowering the entry barrier by decreasing the computing requirements. By introducing a structure containing multiple specialized models rather than having general-purpose models the possibility of using smaller and lighter models can increment the number of compatible devices. To amplify the effect of this methodology the use of fine-tuned, engine-specific models can allow the production of increasingly better results in the context of scripting for specific applications. This improvement would also signify a drastic decrease of the knowledge required by the user to create the specific implementation's design, allowing to focus on the high level logic rather than the technical, engine-dependent details.

Bibliography

[1] Ollama. https://aiwiki.ai/wiki/ollama, 2026. Updated June 20, 2026.

[2] Prasanth Aby T. Alibaba rolls out new AI coding model Qwen3-Coder, says it's their most powerful. https://www.computerworld.com/article/4027149/alibaba-rolls-out-new-ai-coding-model-qwen3-coder-says-its-their-most-powerful.html, 2025.

[3] Adobe. Agile methodology: Frameworks and best practices, 2025.

[4] Adobe. Waterfall methodology – project management, 2025.

[5] Sulaiman Aftan and Habib Shah. A survey on BERT and its applications. In *2023 20th Learning and Technology Conference (L&T)*, pages 161–166. IEEE, 2023.

[6] Saiqa Aleem, Luiz Fernando Capretz, and Faheem Ahmed. Game development software engineering process life cycle: a systematic review. *Journal of Software Engineering Research and Development*, 4(1):6, Nov 2016.

[7] Alibaba Cloud. Alibaba's open source AI journey: Innovation, collaboration, and future visions. https://www.alibabacloud.com/blog/alibabas-open-source-ai-journey-innovation-collaboration-and-future-visions_602026, 2024.

[8] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Susmita Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens tastes Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners, 2020.

[9] Diane R. Caballar and C. Stryker. What is AI code refactoring?, 2026.

[10] Dake Chen, Haoyang Zhang, Hanbin Wang, Yunhao Huo, Yuzhao Li, and Junjie Wang. GameGPT: Multi-agent collaborative framework for game development, 2025.

[11] Chrislb. Recurrent layer neural network. Wikimedia Commons, 2026. Available under CC BY-SA 3.0 license (http://creativecommons.org/licenses/by-sa/3.0/).

[12] Henrik Engström, Jenny Brusk, and Patrik Erlandsson. Prototyping tools for game writers. *The Computer Games Journal*, 7(3):153–172, Sep 2018.

[13] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. PAL: Program-aided language models, 2023.

[14] Daniel Voigt Godoy. BERT next sequence prediction task. Wikimedia Commons, 2026. Available under CC BY 4.0 license (https://creativecommons.org/licenses/by/4.0).

[15] Crina Grosan. *Intelligent Systems: A Modern Approach*. Springer-Verlag Berlin Heidelberg, Berlin, Heidelberg, 2011.

[16] John K. Haas. A history of the Unity game engine. 2014.

[17] Amna Hassan. Automated Unity game template generation from GDDs via NLP and multi-modal LLMs, 2025.

[18] Chaitra Hegde and Shrikumar Patil. Unsupervised paraphrase generation using pre-trained language models, 2020.

[19] Jordan Hoffmann, Sebastian Borgeaud, Arthur converters Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models, 2022.

[20] IBM. What is agentic architecture? https://www.ibm.com/think/topics/agentic-architecture, 2026.

[21] Yuening Jia. The transformer model architecture. Wikimedia Commons, 2026. Available under CC BY-SA 3.0 license (https://creativecommons.org/licenses/by-sa/3.0).

[22] Oğuzhan Karahan. The evolution of Qwen: From inception to today. https://618media.com/en/blog/the-evolution-of-qwen-from-inception-to-today/#key-features-and-capabilities-of-qwen, mar 2025. 618Media Blog.

[23] Linux-Server-Admin Wiki. Ollama history. https://wiki.linux-server-admin.com/web-apps/genai/ollama/history, 2026.

[24] Xinyu Mao, Wanli Yu, Kazunori D. Yamada, and Michael R. Zielewski. Procedural content generation via generative artificial intelligence, 2024.

[25] MarioWiki Contributors. New super Mario bros. E3 2005 kit cover artwork. MarioWiki, 2005. Fair use cover.

[26] MarioWiki Contributors. New super Mario bros. gameplay screenshot. MarioWiki, 2006. Fair use screenshot.

[27] MarioWiki Contributors. New super Mario bros. in-game artwork/screenshot. MarioWiki, 2006. Fair use screenshot.

[28] David A. Medler. A brief history of connectionism. *Neural Computing Surveys*, 1:61–101, 1998.

[29] Niket Mehta. The role of AI in game development and player experience. In *Proceedings of the International Conference on Innovative Computing & Communication (ICICC 2024)*, 2025.

[30] Tauheed Khan Mohd, Fernando Bravo-Garcia, Landen Love, Mansi Gujadhur, and Jason Nyadu. Analyzing strengths and weaknesses of modern game engines. *International Journal of Computer Theory and Engineering*, 15(1):54–60, 2023.

[31] Mungomash LLC. Qwen versions. https://mungomash.com/ai/qwen/versions/, 2026. Accessed: 2026-07-09.

[32] Cosmia Nebula. Long short-term memory cell, 1997 version. Wikimedia Commons, 2026. Available under CC BY-SA 4.0 license (https://creativecommons.org/licenses/by-sa/4.0).

[33] OpenAI. Better language models and their implications, 2019.

[34] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F. Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback, 2022.

[35] Joon Sung Park, Joseph C. O'Brien, Meredith Ringel Cai, Carrie J. Morris, Percy Liang, and Michael S. Bernstein. Example of a generative agent architecture. Wikimedia Commons, 2026. Available under CC BY-SA 4.0 license (https://creativecommons.org/licenses/by-sa/4.0).

[36] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model, 2024.

[37] Pranab Sahoo, Ayush Kumar Singh, Sriparna Saha, Vinija Jain, Samrat Mondal, and Aman Chadha. A systematic survey of prompt engineering in large language models: Techniques and applications, 2025.

[38] Hojjat Salehinejad, Sharan Sankar, Joseph Barfett, Errol Colak, and Shahrokh Valaee. Recent advances in recurrent neural networks. *arXiv preprint arXiv:1801.01078*, 2017.

[39] Nisan Stiennon, Long Ouyang, Jeff Wu, Daniel M. Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul F. Christiano. Learning to summarize from human feedback, 2022.

[40] Shyam Sudhakaran, Miguel González-Duque, Claire Glanois, Matthias Freiberger, Elias Najarro, and Sebastian Risi. MarioGPT: Open-ended text2level generation through large language models, 2023.

[41] Treccani. SISTEMI ESPERTI enciclopedia italiana - V appendice (1994), 1994.

[42] Unity Technologies. MonoBehaviour. https://docs.unity3d.com/6000.3/Documentation/Manual/class-MonoBehaviour.html, 2026. Unity 6000.3 Documentation.

[43] Unity Technologies. ScriptableObject. https://docs.unity3d.com/6000.3/Documentation/Manual/class-ScriptableObject.html, 2026. Unity 6000.3 Documentation.

[44] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.

[45] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models, 2023.

[46] Tianyu Wu, Shizhu He, Jingping Liu, Siqi Artis Sun, Kang Liu, Qing-Long Han, and Yang Tang. A brief overview of ChatGPT: The history, status quo and potential future development. *IEEE/CAA Journal of Automatica Sinica*, 10(5):1122–1136, 2023.

[47] Yifan Zhang, Yang Yuan, and Andrew Chi-Chih Yao. Meta prompting for AI systems, 2025.